

行程表 (暫定)

W	Date	Lecture	Homework	TA Class
1	09/10, 09/11	Course Information / No class - Registration	Homework 00 (Wed.)	V
2	09/17, 09/18	Git Introduction		
3	09/24, 09/25	OOP Concept	Homework 01 (Wed.)	V
4	10/01, 10/02	Introducing What C++ is		
5	10/08, 10/09	Class / Essential STL Introduction	Homework 02 (Wed.)	
6	10/15, 10/16	Lec03: Encapsulation		V
7	10/22, 10/23	Lec04: Inheritance	Homework 03 (Wed.)	
8	10/29, 10/30	Lec04: Inheritance		V
9	11/05, 11/06	Physical Hand-Written Midterm	Homework 04 (Mon.)	
10	11/12, 11/13	Physical Computer-based Midterm		
11	11/19, 11/20	Lec05: Polymorphism	Homework 05 (Wed.)	V
12	11/26, 11/27	Lec05: Polymorphism		
13	12/03, 12/04	Lec05: Polymorphism - LAB	Homework 06 (Wed.)	V
14	12/10, 12/11	Lec06: Composition & Interface		
15	12/17, 12/18	Lec06: Composition & Interface	Homework 07 (Wed.)	
16	12/24, 12/25	Lec06: Composition & Interface - Lab/ No class		V
17	12/31, 01/01	Physical Hand-Written Final / No class		
18	01/07, 01/08	Lec07: Experience sharing & OOPL - Preparation / Physical Computer-based Final		

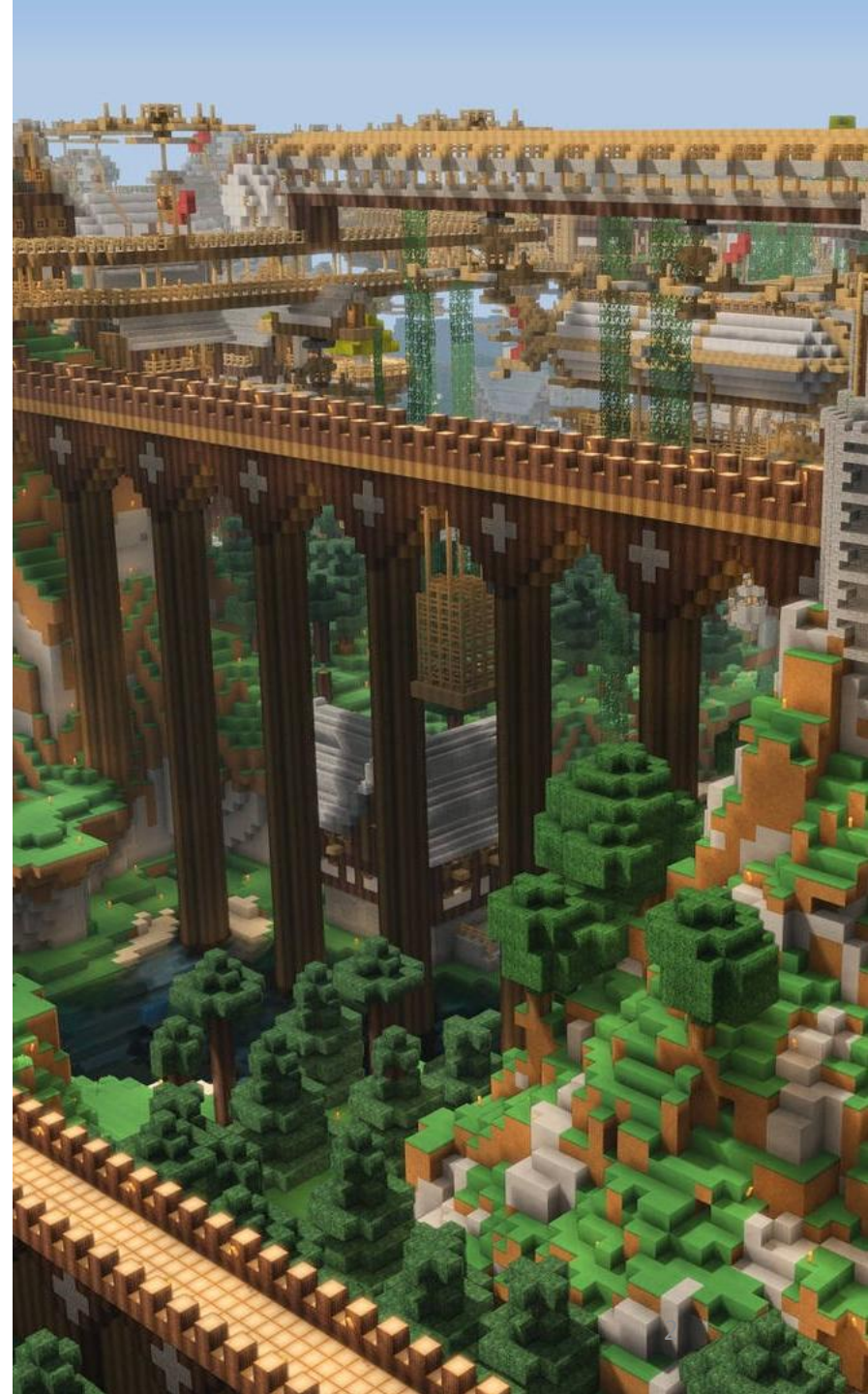
物件導向程式設計

第捌章：組合與介面

授課講師：孫勤昱博士

國立臺北科技大學 資訊工程系

cysun@ntut.edu.tw



大綱

- Schedule Review
- 回顧：組合大概概念
- 物件委派 (Object Delegation)
- 聚合 (Aggregation)
- 回顧：介面大概概念
- 定義物件導向中的耦合 (Coupling)
- 論及耦合 (Coupling)
- 利用組合與介面取代繼承類別
- 組合與介面用於解決什麼樣的問題？
 - The Diamond Problem
 - Combining Interface
 - Template Issue
 - The suggestion of composition, interface, and inheritance.
- Dependency Injection



回顧：組合大概概念

透過之前講的 Minecraft 來回顧組合

- 我們在講的組合 (Composition) 不是數學上的組合 (Combination)

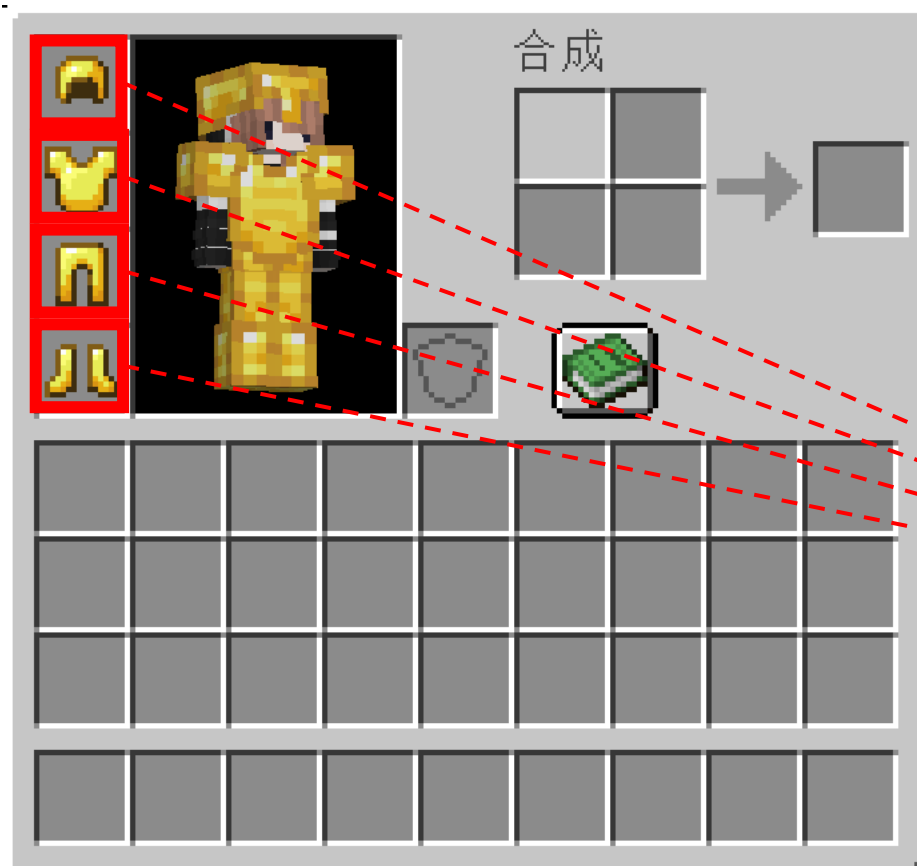
喬喜蛋炒飯
專賣店

專賣店

地址：台北市臨沂街3巷7號
：(02)23963554 0936260601
AM11:00-PM2:00 PM5:00-PM8:00
一個建築師可外送

回顧：組合大概念

- 對於主角，他們具有許多裝備，例如頭盔、衣服、褲子、鞋子等等
- 透過這些裝備，能夠提升角色的防禦值等等的屬性



回顧：組合大概念

- 可以發現「一個角色」，基本上可以透過不同的頭盔、衣服、褲子與鞋子等裝備來搭配出不同的屬性值



皮革套裝

 × 3.5



金套裝

 × 5.5



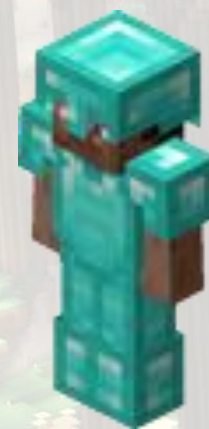
鎖鏈套裝

 × 6



鐵套裝

 × 7.5



鑽石套裝

 × 10

註：實際上，遊戲畫面上的盔甲數量不等於遊戲設定防禦值，而是要乘上2

回顧：組合大概念

- 每個角色 (Character) 會穿戴 4 件防具
 - 頭盔 (helmet)
 - 胸甲 (chestplate)
 - 護腿 (leggings)
 - 靴子 (boots)
- 每一件防具有兩個屬性
 - 名稱：例如 "Iron Helmet" (字串)
 - 防禦力：例如 5.0 (浮點數)
- 請設計一個 Character 類別，完成以下要求：
 - 在 private 區塊中，宣告足夠的成員變數來儲存：
 - 四件防具的名稱 (字串)
 - 四件防具的防禦力 (浮點數)
 - 實作一個建構子 Character() 參數中要能接受這四件防具的名稱與防禦力，並把它們存到成員變數中
 - 實作一個成員函式：GetDefense() 來回傳這個角色總防禦力 (四件防具防禦力的加總)



頭盔



衣服



褲子



鞋子



法師

回顧：組合大概概念

- 分析程式碼



頭盔



衣服



法師



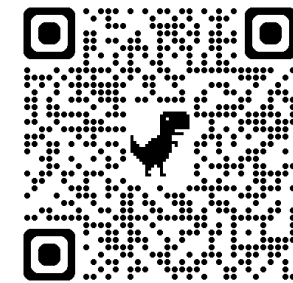
褲子



鞋子



```
4 class Character {
5     private:
6         std::string helmetName;
7         float helmetDefense;
8         std::string chestplateName;
9         float chestplateDefense;
10        std::string leggingsName;
11        float leggingsDefense;
12        std::string bootsName;
13        float bootsDefense;
14
15    public:
16        Character(
17            std::string hName,
18            float hDefense,
19            std::string cName,
20            float cDefense,
21            std::string lName,
22            float lDefense,
23            std::string bName,
24            float bDefense)
25        : helmetName(hName), helmetDefense(hDefense),
26          chestplateName(cName), chestplateDefense(cDefense),
27          leggingsName(lName), leggingsDefense(lDefense),
28          bootsName(bName), bootsDefense(bDefense) {}
29
30        float GetDefense() {
31            return helmetDefense +
32                   chestplateDefense +
33                   leggingsDefense +
34                   bootsDefense;
35        }
36    };
```



程式碼這樣寫好嗎？

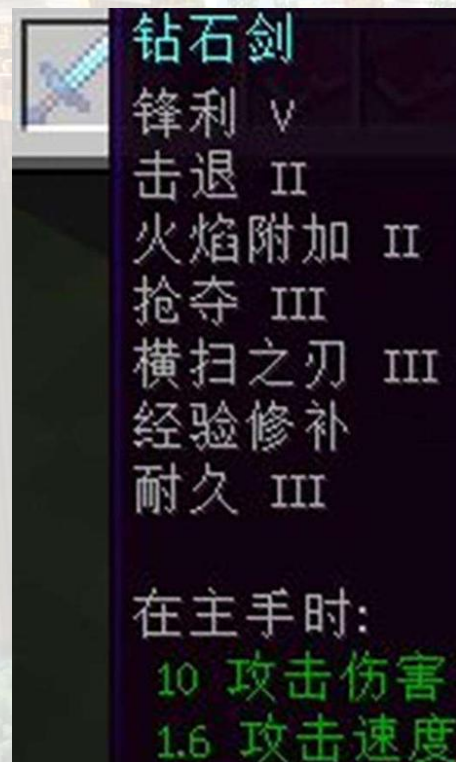
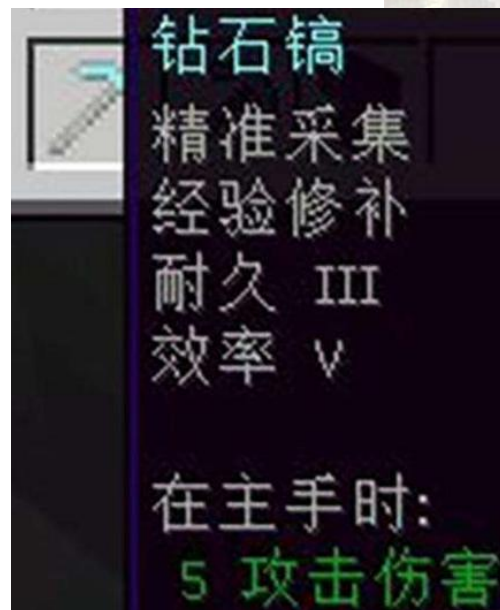
回顧：組合大概概念

- 觀察一：道具的屬性複雜
 - 如左圖



- 類別內容會因為屬性而變得混亂

```
1 class Character {  
2     // 原本code內容  
3  
4     // 特殊功能：加速鞋  
5     void SpeedBoots() {  
6         if (bootsName == "SpeedBoots_Boots") {  
7             // 實現複雜的加速功能  
8         }  
9     }  
10 };
```



回顧：組合大概念

- 觀察二：裝備的種類複雜化
 - Minecraft 裝備只有四個部位：頭盔、衣服、褲子與鞋子
- 如果開始要複雜化裝備系統...
 - 雙武器
 - 腰帶
 - 手鐲 × 2
 - 耳環 × 2
 - 項鍊
 - 戒子 × 2
 - 翅膀
 - 坐騎
 - 寵物
 - ...



回顧：組合大概念

- 在這個觀點上，我們可以把頭盔、衣服、褲子與鞋子當作角色的成員
- 我們可以初始化這些頭盔、衣服、褲子與鞋子，運算這些裝備來得到防禦值 5.5



頭盔



衣服



法師



褲子



鞋子



```
4 class Character{  
5     private:  
6         Helmet helmet;  
7         Chestplate chestplate;  
8         Leggings leggings;  
9         Boots boots;  
10  
11     public:  
12         Character(Helmet helmet,  
13                   Chestplate chestplate,  
14                   Leggings leggings,  
15                   Boots boots){}  
16 };
```


回顧：組合大概概念

- 我們稱這個作法叫做「組合」：**成員內具有其他的類別成員**
- 我們會在建構子或者 Setter 中設定這些成員



頭盔



法師



褲子



衣服



鞋子



```
4 class Character{
5     private:
6         Helmet helmet;
7         Chestplate chestplate;
8         Leggings leggings;
9         Boots boots;
10
11     public:
12         Character(Helmet helmet,
13                 Chestplate chestplate,
14                 Leggings leggings,
15                 Boots boots){}
16
17         int GetDefense(){
18             return helmet.GetDefense() +
19                    chestplate.GetDefense() +
20                    leggings.GetDefense() +
21                    boots.GetDefense();
22         }
23     };
```

回顧：組合大概概念

- 透過組合，我們可以真的讓某個成員是一個類別，取代大量的成員來描述一種類別

```
4 class Character {
5     private:
6         std::string helmetName;
7         int helmetDefense;
8         std::string chestplateName;
9         int chestplateDefense;
10        std::string leggingsName;
11        int leggingsDefense;
12        std::string bootsName;
13        int bootsDefense;
14
15    public:
16        Character(
17            std::string hName,
18            int hDefense,
19            std::string cName,
20            int cDefense,
21            std::string lName,
22            int lDefense,
23            std::string bName,
24            int bDefense)
25        : helmetName(hName), helmetDefense(hDefense),
26          chestplateName(cName), chestplateDefense(cDefense),
27          leggingsName(lName), leggingsDefense(lDefense),
28          bootsName(bName), bootsDefense(bDefense) {}
29
30        int GetDefense() {
31            return helmetDefense +
32                   chestplateDefense +
33                   leggingsDefense +
34                   bootsDefense;
35        }
36    };
```



```
4 class Character{
5     private:
6         Helmet helmet;
7         Chestplate chestplate;
8         Leggings leggings;
9         Boots boots;
10
11    public:
12        Character(Helmet helmet,
13                  Chestplate chestplate,
14                  Leggings leggings,
15                  Boots boots){}
16
17        int GetDefense(){
18            return helmet.GetDefense() +
19                   chestplate.GetDefense() +
20                   leggings.GetDefense() +
21                   boots.GetDefense();
22        }
23    };
```


回顧：組合大概概念

- 由於裝備被使用者「擁有」，所以如果這個玩家死掉之後，那麼裝備也應該一起消失
- 在 Destructor 中，我們將這個類別擁有的成員全部釋放



頭盔



衣服



你



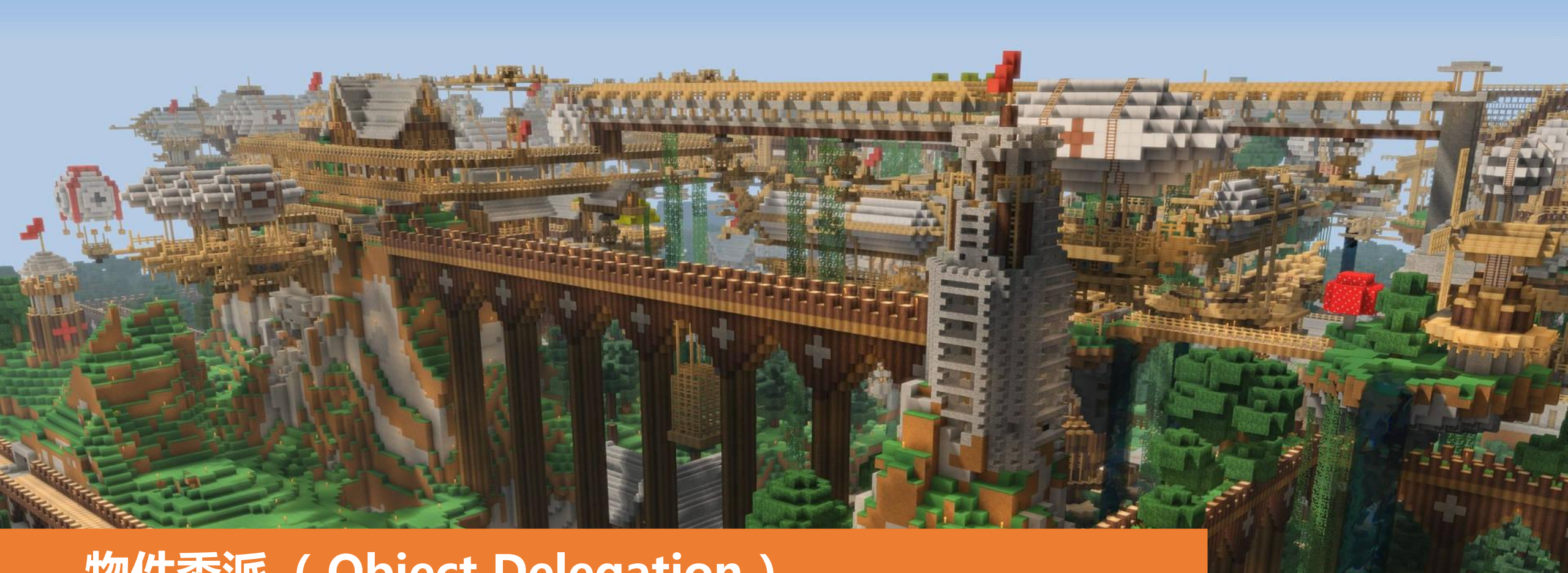
褲子



鞋子

- 實際上 Minecraft 遊戲設計是噴裝，我們先假設遊戲是「極限」模式



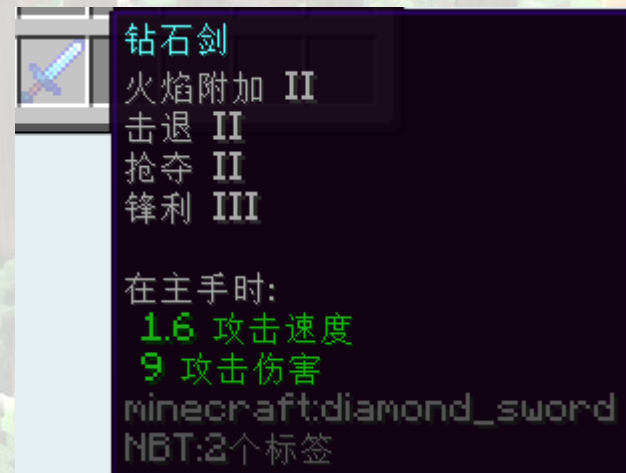


物件委派 (Object Delegation)

我們將描述組合的其中一個應用：物件委派

物件委派 (Object Delegation)

- 組合可以做類似「委託」的事情
- 在 Minecraft 裡面有個道具叫做附魔台，你可以把武器跟青金石放入，就能夠強化武器



物件委派 (Object Delegation)

- 實作程式碼如下：

```
6 int main()
7 {
8     EnhancementTable enhancementTable; ----- ① 創附魔台
9     Equipment equipment;
10
11     std::vector<std::string> enhancements = enhancementTable.GetEnhancements(); ----- ② 取得附魔屬性
12     equipment.AddEnhancement(enhancement[0]); ----- ③ 把附魔屬性加到裝備上
13
14     return 0;
15 }
```

- 我們可以透過創建附魔台類別，然後取得裡面可以附魔的種類，最後加到裝備上
- 這樣是不是有點奇怪？
 - 很像是我們今天是取得了某個附魔的物品，然後加到裝備上
 - 上述這件事情，是不是應該要由「附魔台」來幫我們完成比較好？

物件委派 (Object Delegation)

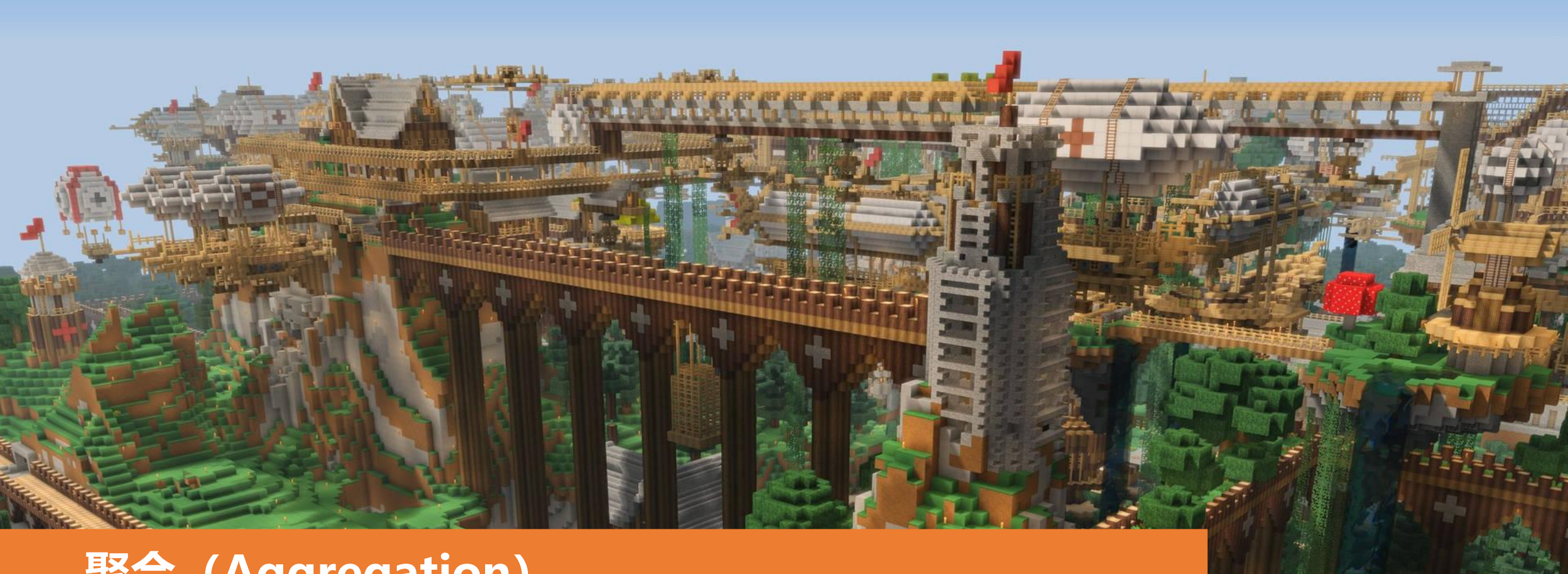
- 如果我們用右邊的方式來創建附魔台：
- 我們將裝備委派到了這個 EnhancementTable 類別，並透過 EnhanceEquipment(int index) 來幫裝備增加附魔，使裝備得到能力提升
- 基本上裝備是被委派到了類別裡面做處理，而非我們將 EnhancementTable 中的附魔拿出來並加到裝備上
- 更加直觀，符合遊戲運作的邏輯

```
4 class EnhancementTable{
5     private:
6         Equipment equipment;
7         int LapisLazuliCount;
8
9     public:
10        EnhancementTable() = default;
11
12        //取得附魔台中的附魔
13        std::vector<std::string> GetEnhancements(){
14            //實作 "取得附魔台中的附魔" 這個功能的細節
15        }
16
17        //設定待附魔的裝備與青金石
18        void SetEquipment(Equipment equipment, int lapisLazuliCount){
19            this->equipment = equipment;
20            this->LapisLazuliCount = lapisLazuliCount;
21        }
22
23        //附魔裝備，選擇某個附魔來附魔到裝備
24        Equipment EnhanceEquipment(int index){
25            equipment.AddEnhancement(GetEnhancements()[index]);
26            return equipment;
27        }
28    };
```

物件委派 (Object Delegation)

- 透過委派這個裝備進入 EnhancementTable 來進行裝備的附魔，是透過組合能夠帶來的設計方法
- 這個技巧通常稱為物件委派 (Object Delegation)

```
4 class EnhancementTable{
5     private:
6         Equipment equipment;
7         int LapisLazuliCount;
8
9     public:
10        EnhancementTable() = default;
11
12        //取得附魔台中的附魔
13        std::vector<std::string> GetEnhancements(){
14            //實作 "取得附魔台中的附魔" 這個功能的細節
15        }
16
17        //設定待附魔的裝備與青金石
18        void SetEquipment(Equipment equipment, int lapisLazuliCount){
19            this->equipment = equipment;
20            this->LapisLazuliCount = lapisLazuliCount;
21        }
22
23        //附魔裝備，選擇某個附魔來附魔到裝備
24        Equipment EnhanceEquipment(int index){
25            equipment.AddEnhancement(GetEnhancements()[index]);
26            return equipment;
27        }
28    };
```

聚合 (Aggregation)

聚合是組合的一種衍伸

聚合 (Aggregation)

- 在知道附魔台的概念後，我們面對一個新問題：
 - 如果把附魔台挖掉（移除）的話，裡面的道具會不見嗎？



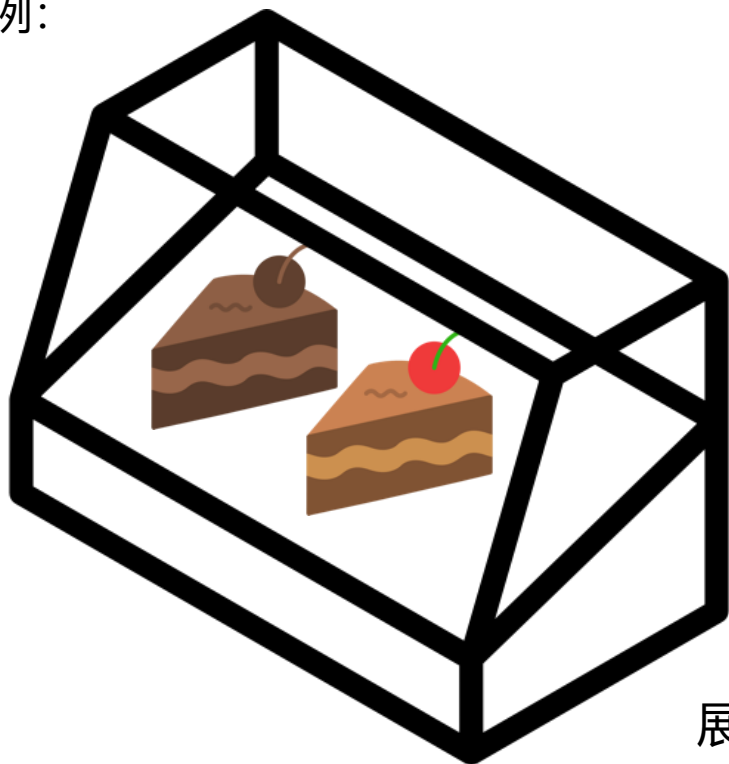
聚合 (Aggregation)

- 可以考慮在使用組合時的兩種情況：
 - 使用**解構子**時，**會**把組合內的成員全部釋放

```
4 class EnhancementTable{
5     private:
6         Equipment equipment;
7         int LapisLazuliCount;
8
9     public:
10        EnhancementTable() = default;
11
12        //取得附魔台中的附魔
13        std::vector<std::string> GetEnhancements(){
14            //實作 "取得附魔台中的附魔" 這個功能的細節
15        }
16
17        //設定待附魔的裝備與青金石
18        void SetEquipment(Equipment equipment, int lapisLazuliCount){
19            this->equipment = equipment;
20            this->LapisLazuliCount = lapisLazuliCount;
21        }
22
23        //附魔裝備，選擇某個附魔來附魔到裝備
24        Equipment EnhanceEquipment(int index){
25            equipment.AddEnhancement(GetEnhancements()[index]);
26            return equipment;
27        }
28 };
```


聚合 (Aggregation)

- 可以考慮在使用組合時的兩種情況：
 - 使用**解構子**時，**不會**把組合內的成員全部釋放
- 為什麼不釋放？
 - 對於一個組合的物件來說，放進去的東西是不是委派，取決於我們應不應該釋放他
- 範例：

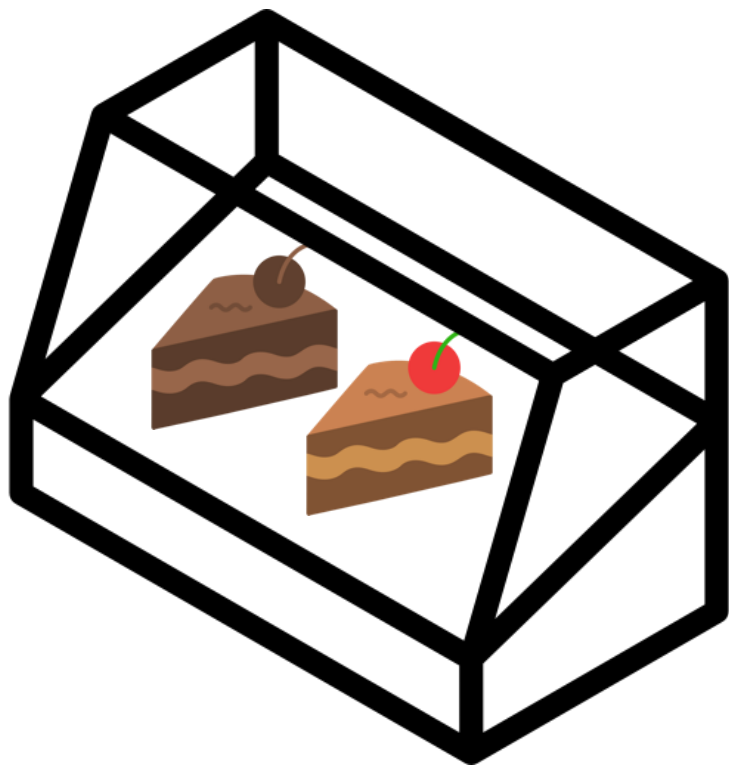


展示櫃與蛋糕 (2022 OOP 的作業 7)

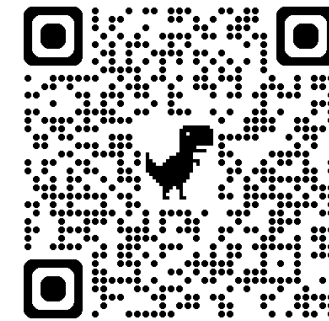


聚合 (Aggregation)

- 當把這個展示櫃拆掉的話，蛋糕應該還要在嗎？
- 實際上，蛋糕應該要在
- 因為展示櫃沒有持有蛋糕，蛋糕僅僅只是放入展示櫃而不是變成展示櫃的一部分



```
5 class Cake{
6     private:
7         std::string name;
8     public:
9         Cake(std::string name): name(name){}
10 };
11
12 class CakeShowcase{
13     private:
14         std::vector<Cake*> cakes;
15         std::string *name;
16     public:
17         CakeShowcase(std::vector<Cake*> cakes){
18             this->cakes = cakes;
19             this->name = new std::string("default");
20         }
21         ~CakeShowcase(){ free(name); }
22 };
23
24 int main(){
25     Cake* cake = new Cake("Cake1");
26     Cake* another_cake = new Cake("Cake2");
27     CakeShowcase* cake_showcase =
28         new CakeShowcase(std::vector<Cake*> {cake, another_cake});
29     std::cout << &cake << std::endl;
30
31
32 }
```



聚合 (Aggregation)

- 範例程式碼
 - 把蛋糕當作類別成員
 - 但不應該釋放類別成員，因為蛋糕指標物件只是被委派進入類別
- 應該要小心 Double Free 的問題
 - 因邏輯上，釋放蛋糕的時機為整個程式碼執行完畢時（非 Showcase這段程式碼！）
 - 若因為 Aggregation 沒處理好，提前釋放蛋糕
 - 就會造成 Double Free 的問題

```
5 class Cake{
6     private:
7         std::string name;
8     public:
9         Cake(std::string name): name(name){}
10 };
11
12 class CakeShowcase{
13     private:
14         std::vector<Cake*> cakes;
15         std::string *name;
16     public:
17         CakeShowcase(std::vector<Cake*> cakes){
18             this->cakes = cakes;
19             this->name = new std::string("default");
20         }
21         ~CakeShowcase(){ free(name); }
22 };
23
24 int main(){
25     Cake* cake = new Cake("Cake1");
26     Cake* another_cake = new Cake("Cake2");
27     CakeShowcase* cake_showcase =
28         new CakeShowcase(std::vector<Cake*> {cake, another_cake});
29     std::cout << "cake" << std::endl;
30     free(cake);
31     free(another_cake);
32 }
```

- 其實 Aggregation 因為有shared pointer 所以就可以被優化了

聚合 (Aggregation)

- 在知道附魔台的概念後，我們面對一個新問題：
 - 如果把附魔台挖掉（移除）的話，裡面的道具會不見嗎？ **NO!**



聚合 (Aggregation)

- 如果這個類別擁有「類別成員」的所有權，我們稱之為組合
 - 在釋放時我們應該要把組合內的成員**全部釋放**，因為成員內的東西是所有物
 - Ex. 如果你讓一台筆電變不見，他的所有零件「電池、螢幕」也應該不見
- 對於這個成員是「委派」給這個類別的，我們稱之為聚合
 - **不應該釋放**，因為東西是委派給這個類別，這個類別自己不見與委派的成員無關
 - Ex. 放入物品的附魔台，在附魔台被破壞時，物品不應該被破壞，因為物品委派給附魔台做附魔



回顧

- 由於裝備被使用者「擁有」，所以如果這個玩家死掉之後，那麼裝備也應該一起消失
- 在 Destructor 中，我們將這個類別擁有的成員全部釋放



頭盔



衣服



你



褲子



鞋子

- 實際上 Minecraft 遊戲設計是噴裝，我們先假設遊戲是「**極限**」模式



回顧：介面大概概念

透過之前講的 Minecraft 來回顧介面

回顧：介面大概概念

- 我們有不同的怪物，這些怪物會在**觸碰**玩家時對玩家造成一定的傷害



殭屍

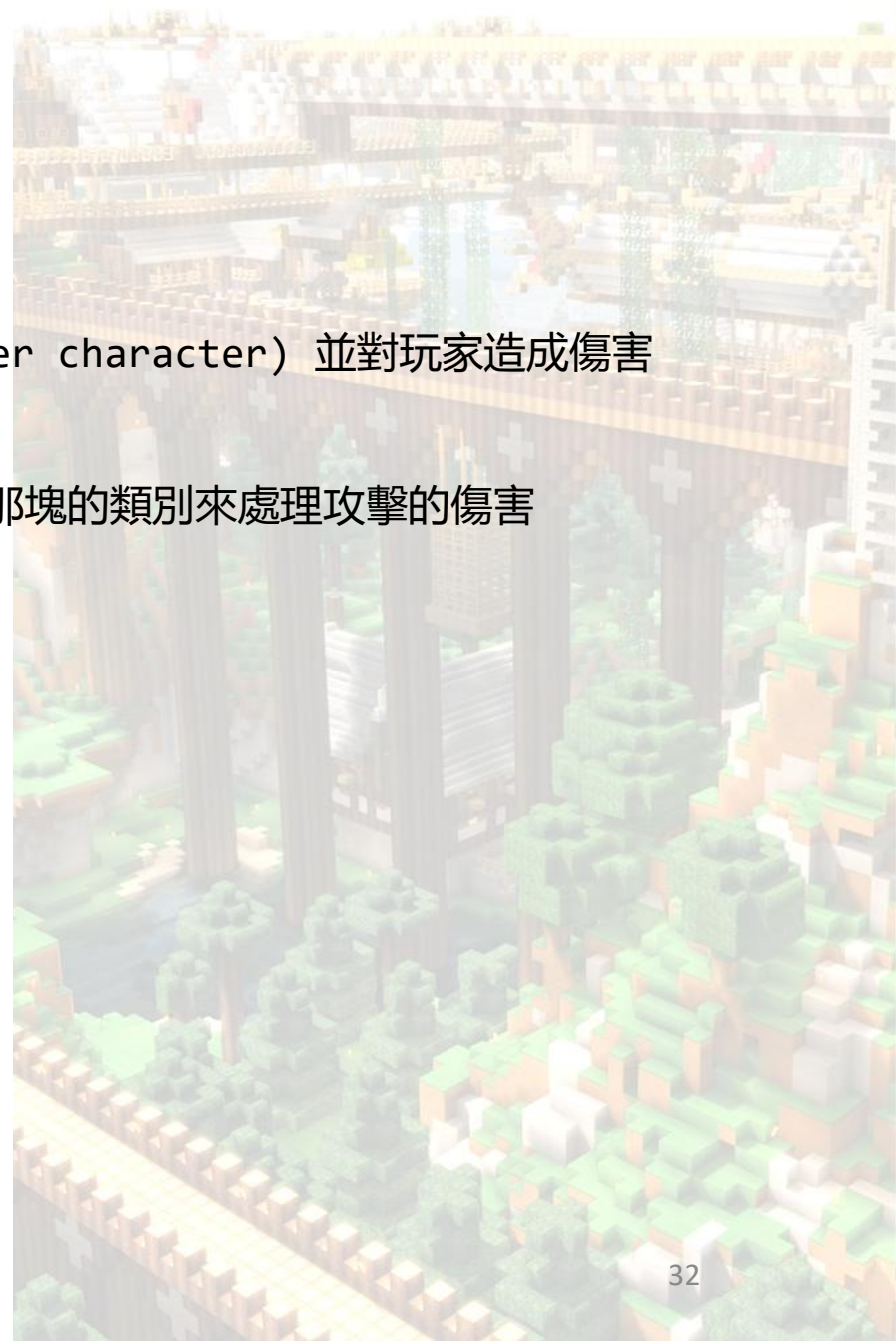


史萊姆

回顧：介面大概概念

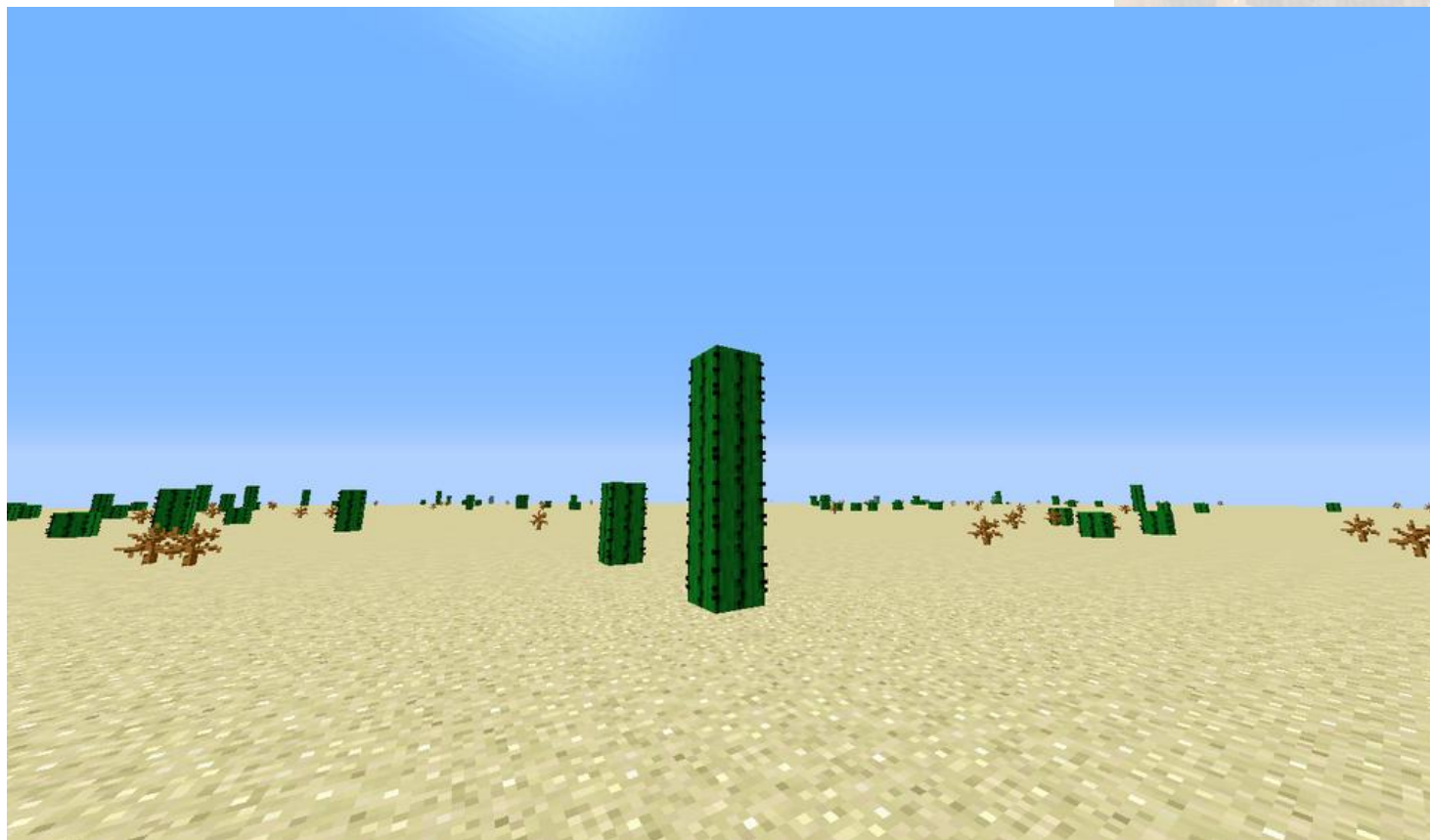
- 這時回過頭來思考一下多型，我們其實可以有一個怪物的雛形，叫做 Mob
- 我們可以讓怪物繼承 Mob，讓 Mob 具有攻擊的 Function 叫 `Attack(Character character)` 並對玩家造成傷害
- 透過多型，我們一樣也能先不管怪物是什麼，但知道他會造成攻擊，呼叫記憶體那塊的類別來處理攻擊的傷害

註：Mob 為一個遊戲術語，指在電玩遊戲中會攻擊玩家的敵對角色



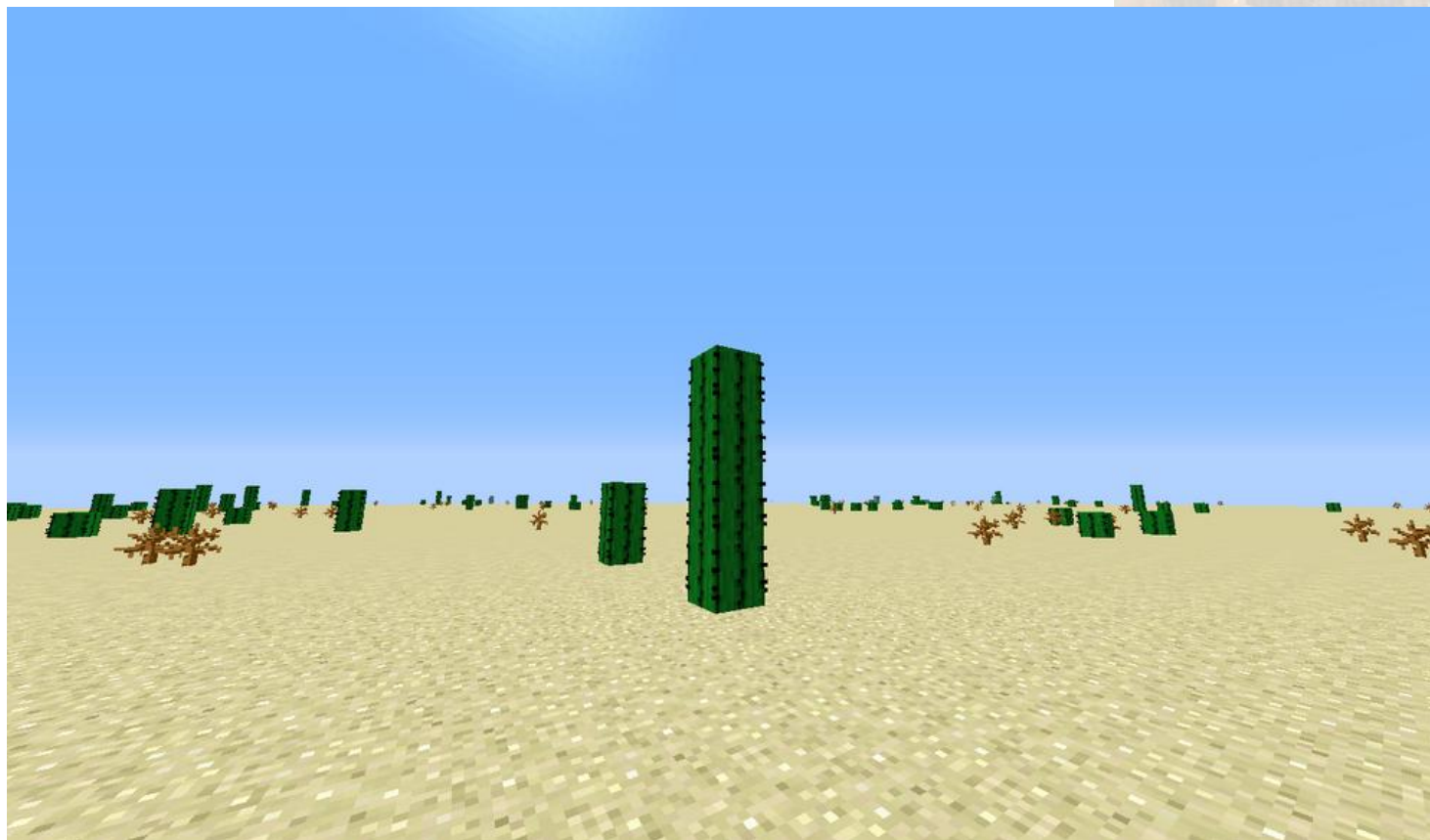
回顧：介面大概念

- 但事實上，不是只有怪物會對使用者造成危害
- Minecraft 有一種植物叫做仙人掌，如果角色觸摸他也會造成傷害



回顧：介面大概念

- 如果我們讓仙人掌也繼承 Mob 並擁有 `Attack(Character character)` 不就行了嗎？
- 實際上可行，但這違反了「繼承是拿來衍伸父類別」的概念，畢竟仙人掌不是一種怪物



回顧：介面大概概念

- 這時候，我們主要是希望「會攻擊玩家的東西」都會有 `Attack(Character character)` 這個**函式**
- 透過這個部分，我們可以考慮使用介面來完成，透過宣告一個 `ICharacterAttackable` 介面
 - 命名通常以 "I" 開頭，表示 Interface

```
class ICharacterAttackable {  
public:  
    virtual void Attack(Character character) = 0;  
};
```

- 由於是 Pure Virtual Function，所以所有繼承這個介面的類別都要實作自己的 `Attack(Character character)`

回顧：介面大概概念

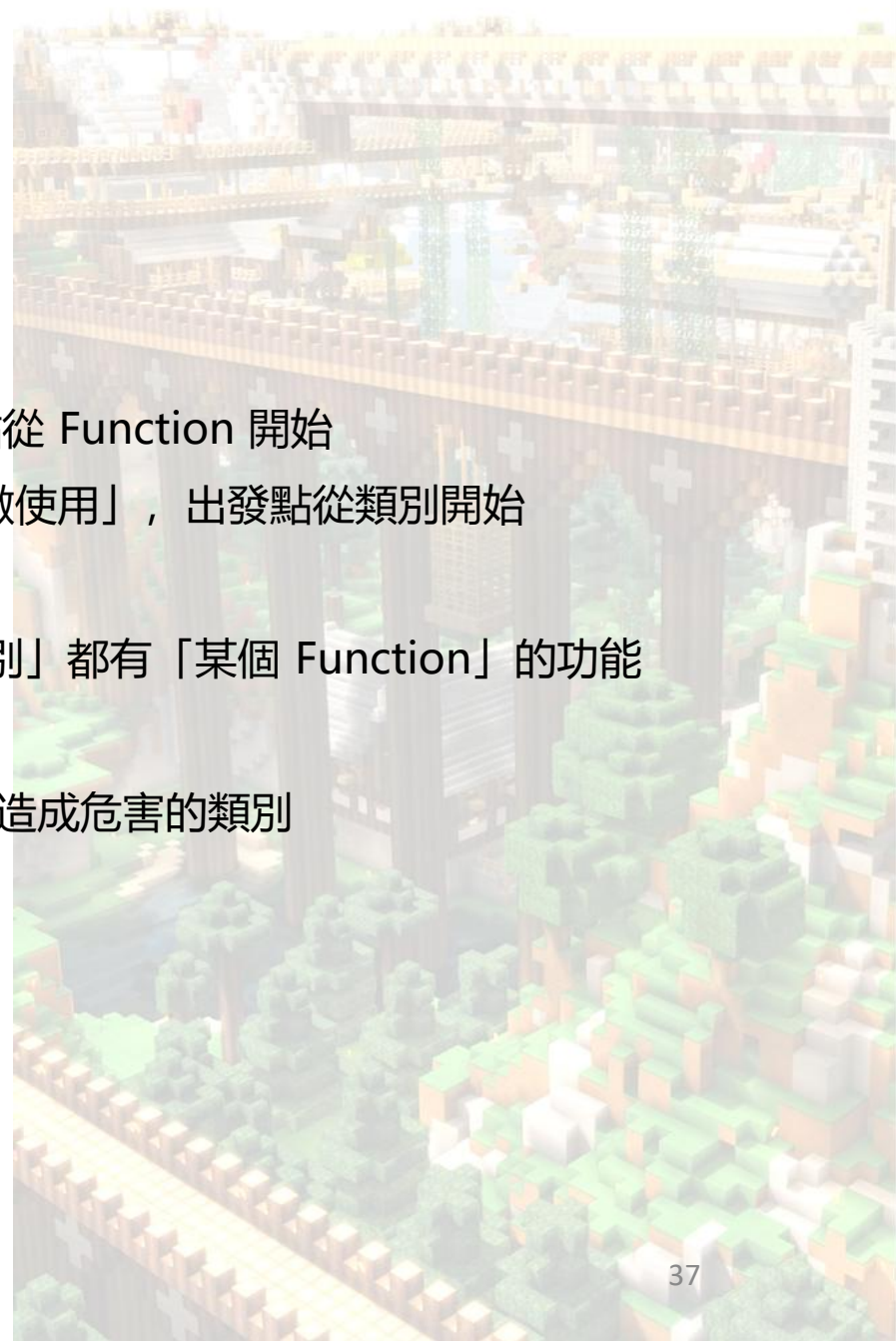
- 那麼我們就能透過這個介面來把所有「會傷害角色」的植物或怪物當作 `ICharacterAttackable`
- 最後呼叫 `Attack(Character character)` 來對玩家造成傷害。

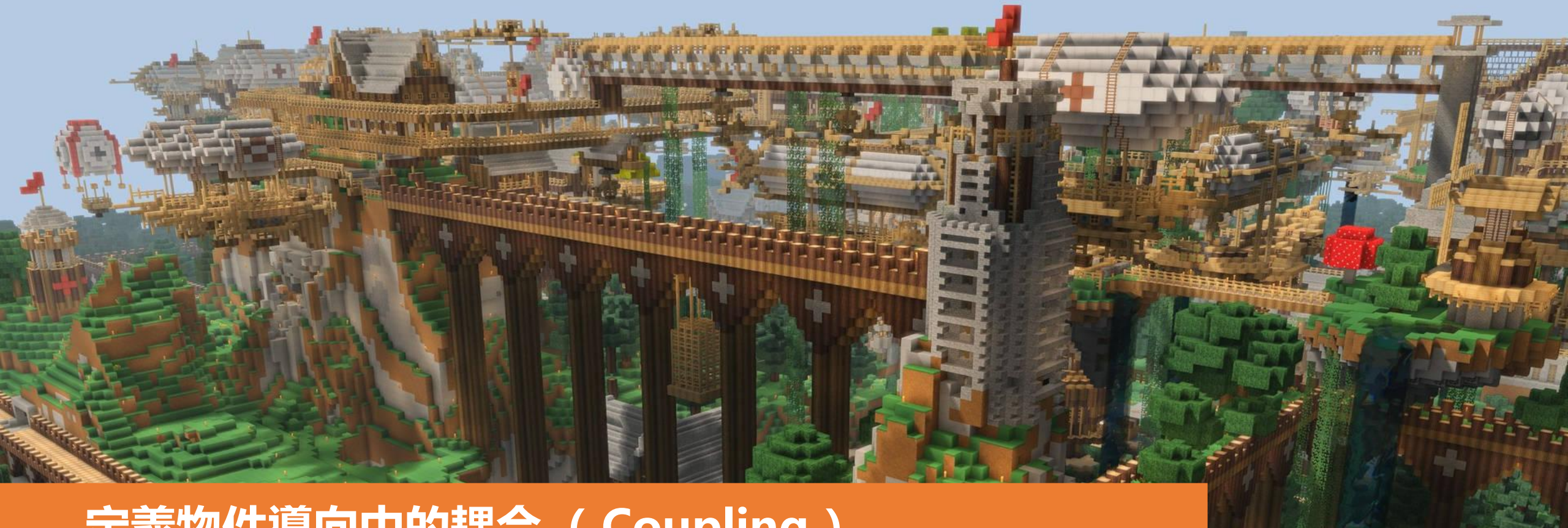
```
int main(){
    Character character;
    std::vector<ICharacterAttackable*> characterAttackableEntities = GetAllCharacterAttackableEntity();

    for(ICharacterAttackable* characterAttackableEntity : characterAttackableEntities){
        characterAttackableEntity->Attack(character);
    }
}
```

回顧：介面大概概念

- 這個與繼承有什麼差別？
- 我們在介面上主要思考的是「會攻擊角色的類別需要有哪些 Function」，出發點從 Function 開始
- 我們在繼承上主要思考的是「這個類別會需要哪些 Function 能夠給底下的子類做使用」，出發點從類別開始
- 在介面上，我們可以不用管底下的子類是不是父類的衍生，而是「這些所有的類別」都有「某個 Function」的功能
 - 所以我們可以不用因為要用繼承，所以需要在意仙人掌是不是一種怪物
 - 但我們可以知道的是無論是仙人掌、殭屍、還是史萊姆，都是一種會對主角造成危害的類別



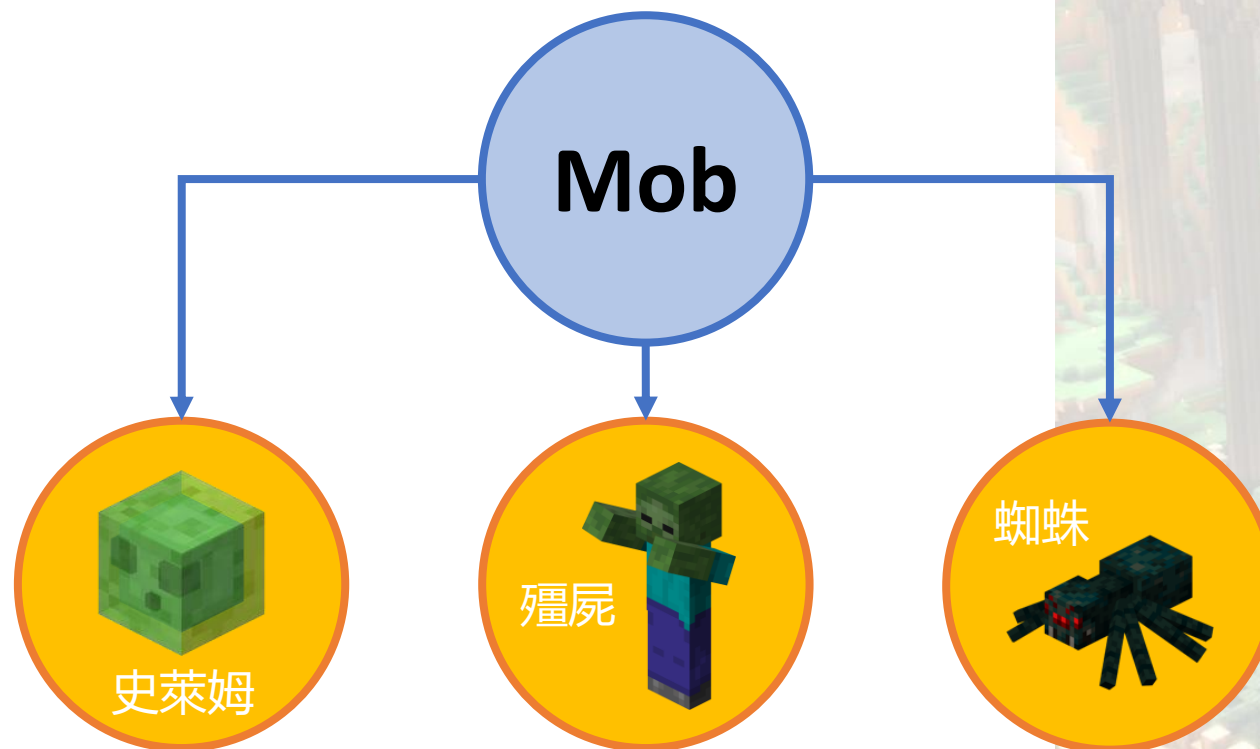


定義物件導向中的耦合（Coupling）

耦合是一種用於描述程式中元件間依賴關係的概念

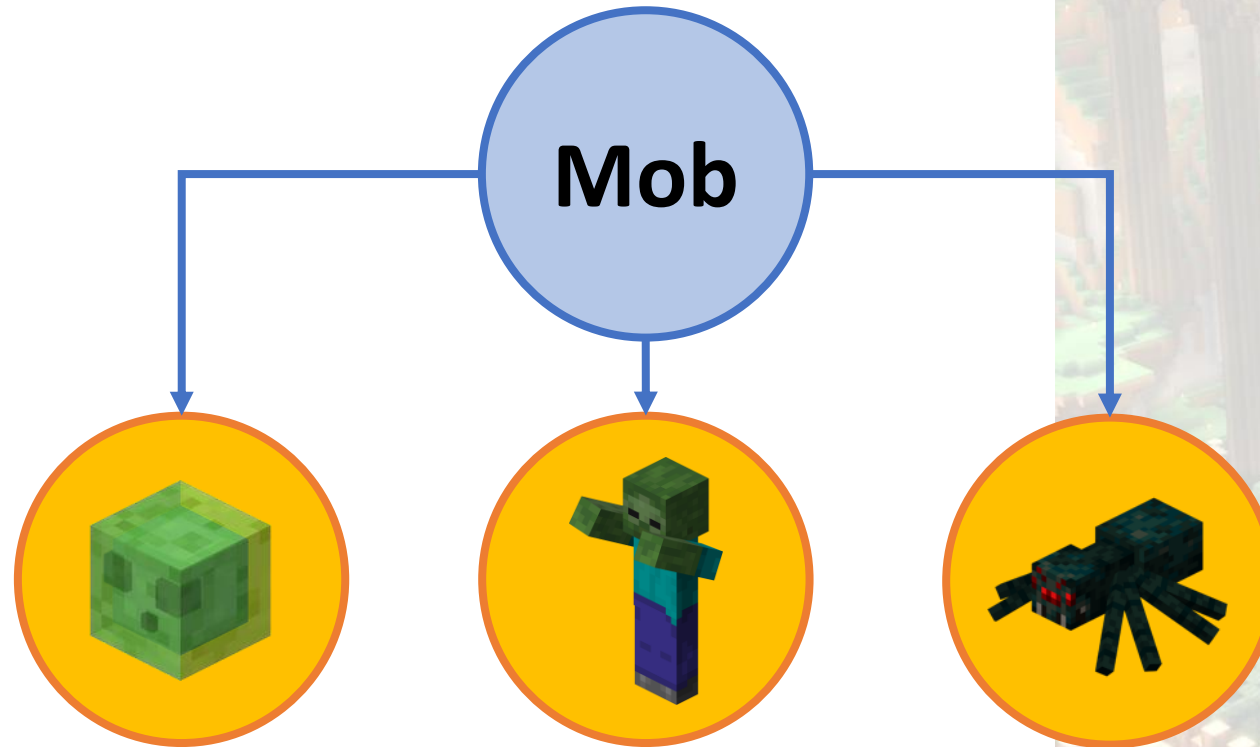
定義物件導向中的耦合 (Coupling)

- 在 Minecraft 中，我們想要先為「史萊姆」、「殭屍」與「蜘蛛」來定義一個父類別，稱為 Mob



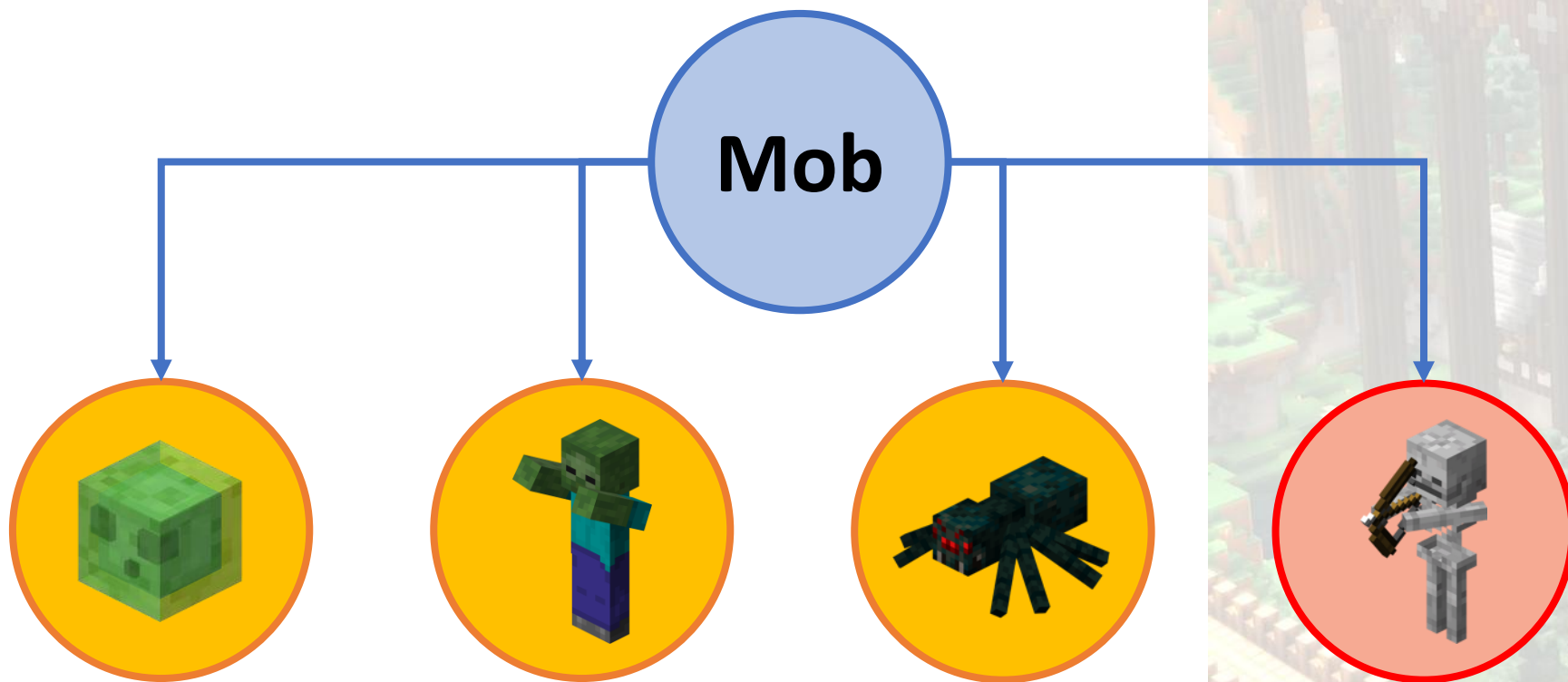
定義物件導向中的耦合 (Coupling)

- 這個 Mob 類別會有怪物要有的一些屬性，一個 Attack (Character character) 方法，用於近戰攻擊使用者



定義物件導向中的耦合 (Coupling)

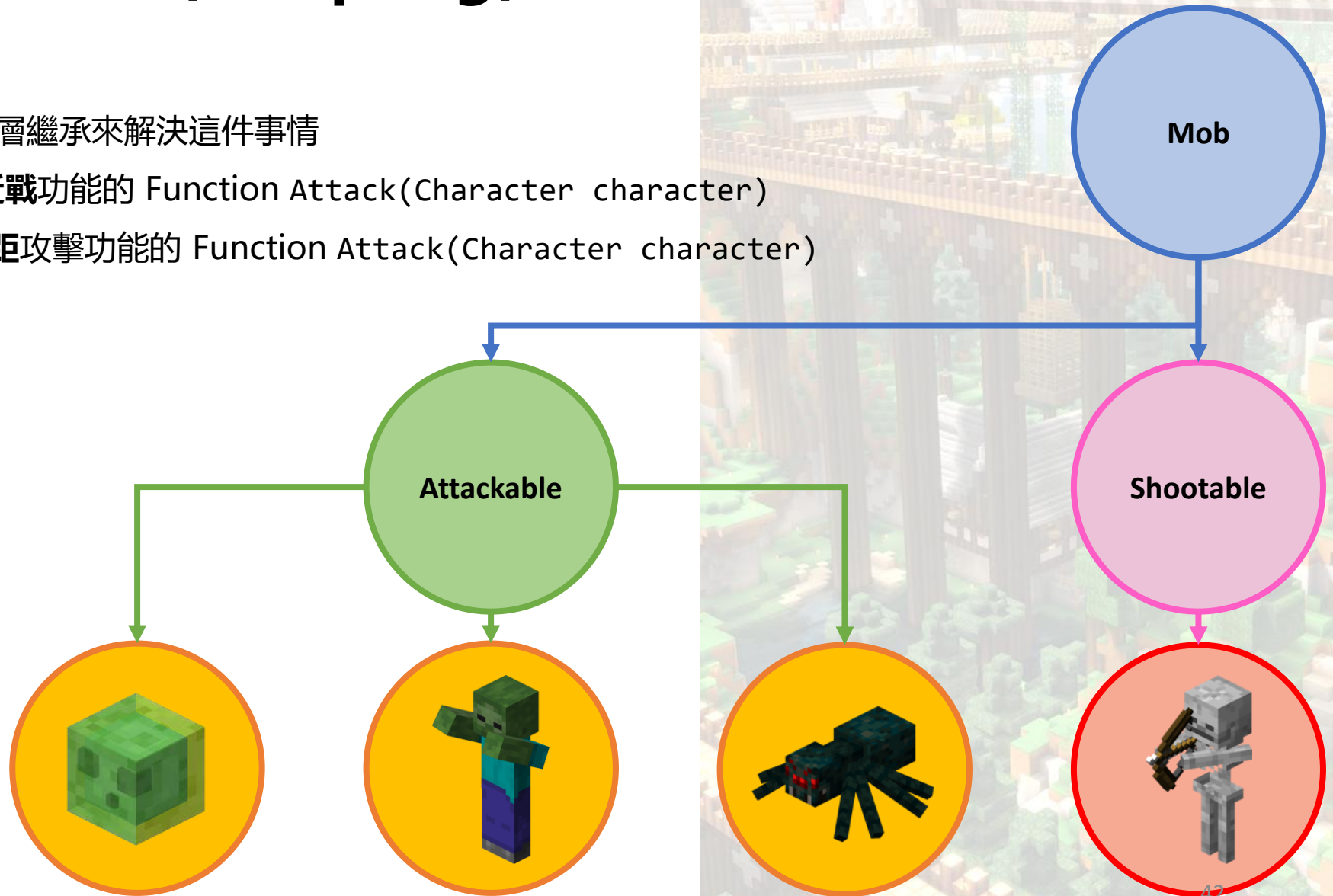
- 如果我們希望有一個「骷髏弓箭手」來加入這個 Mob 成員...



- 讓骷髏弓箭手有近戰攻擊的 Function 其實不太合理，因為骷髏弓箭手會發射弓箭來射傷玩家

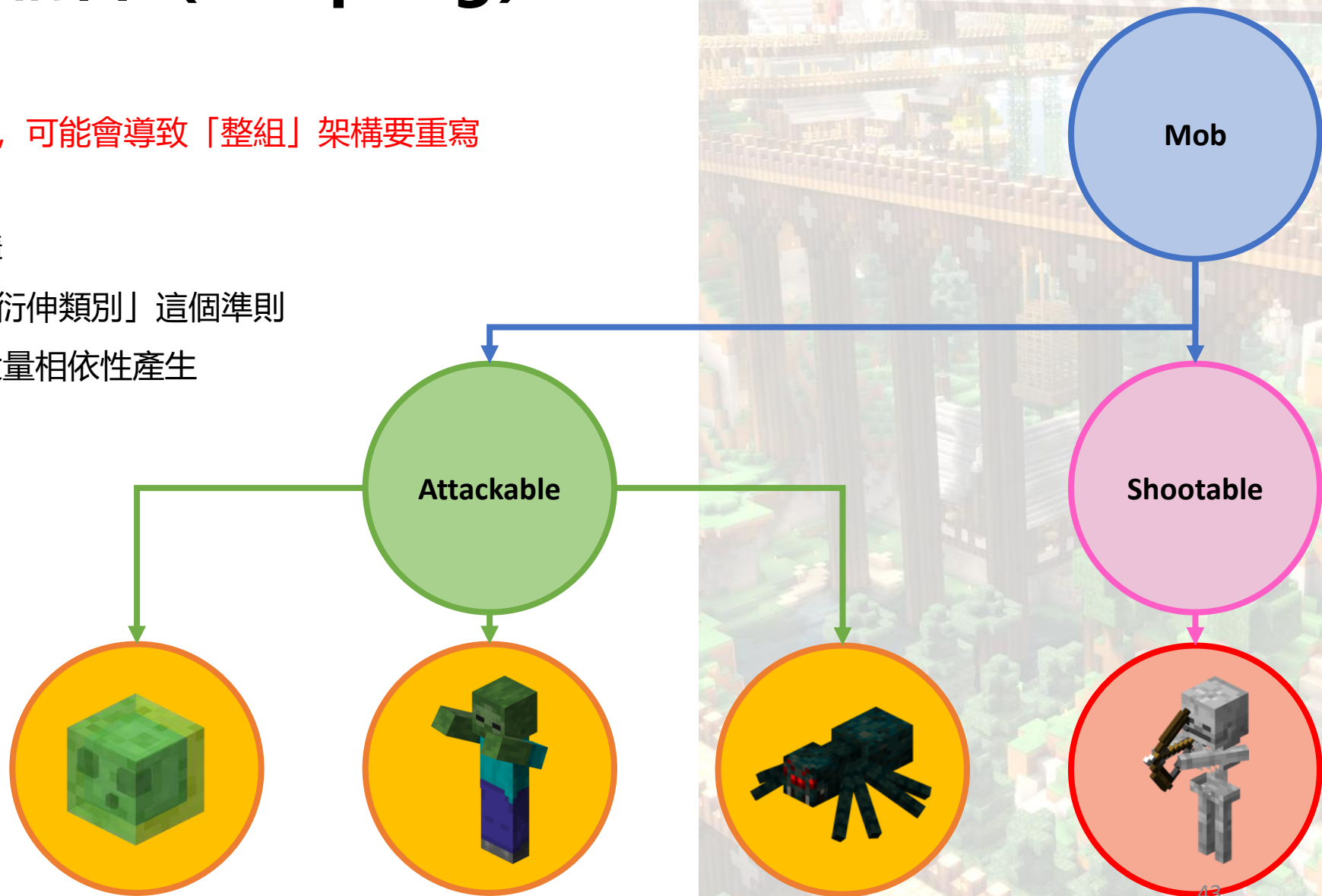
定義物件導向中的耦合 (Coupling)

- 因應這件事情，需要額外加上一層繼承來解決這件事情
 - 讓 Attackable 類別具有**近戰**功能的 Function Attack(Character character)
 - 讓 Shootable 類別具有**遠距**攻擊功能的 Function Attack(Character character)



定義物件導向中的耦合 (Coupling)

- 會發現當新增這個骷髏弓箭手後，可能會導致「整組」架構要重寫
- 這會使「繼承鏈」變長
 - 即為繼承所造成的鏈式結構
- 繼承鏈能夠保證「父類是子類的衍伸類別」這個準則
 - 但是因為這個準則，導致大量相依性產生



定義物件導向中的耦合 (Coupling)

- 所以當新增這個骷髏弓箭手後，可能用於測量 (Measure) 受影響的類別，稱為耦合度
- 像下圖的維護方式可能就是一個**高耦合度**的實作，因為新增一個類別可能導致所有的類別都要更動



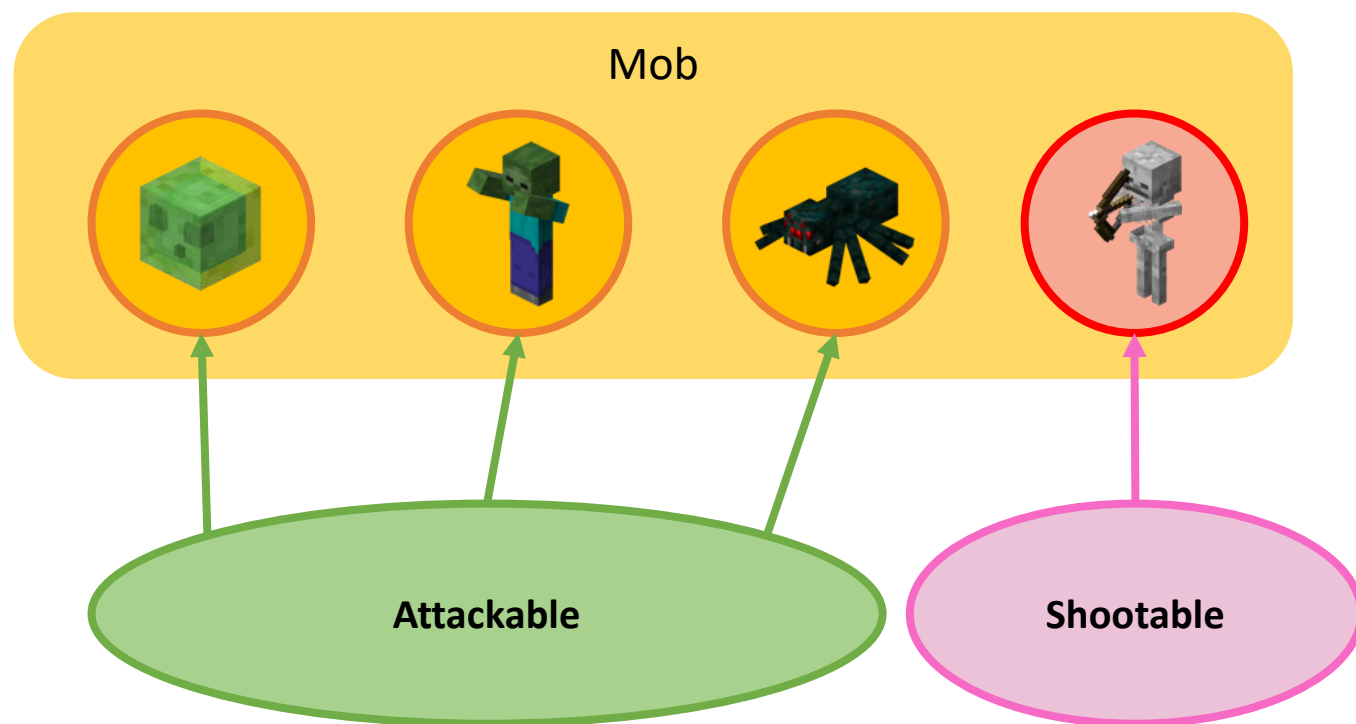
定義物件導向中的耦合（Coupling）

- 耦合其實並不是一個用於精確測量受影響類別數量的方式，而是一種**經驗法則**
 - 「高」耦合度
 - 「低」耦合度
- 我們期望上我們的實作是**低耦合度跟程式碼的重複率做衡量**，可以讓程式的可讀性與可維護性增加
- 為什麼低耦合度的實作有他的好處？



定義物件導向中的耦合 (Coupling)

- 換一個方式來看這個繼承。
- 我們先確保每個怪物都有繼承 Mob，對於每個怪物，再繼承 Attackable 呢？

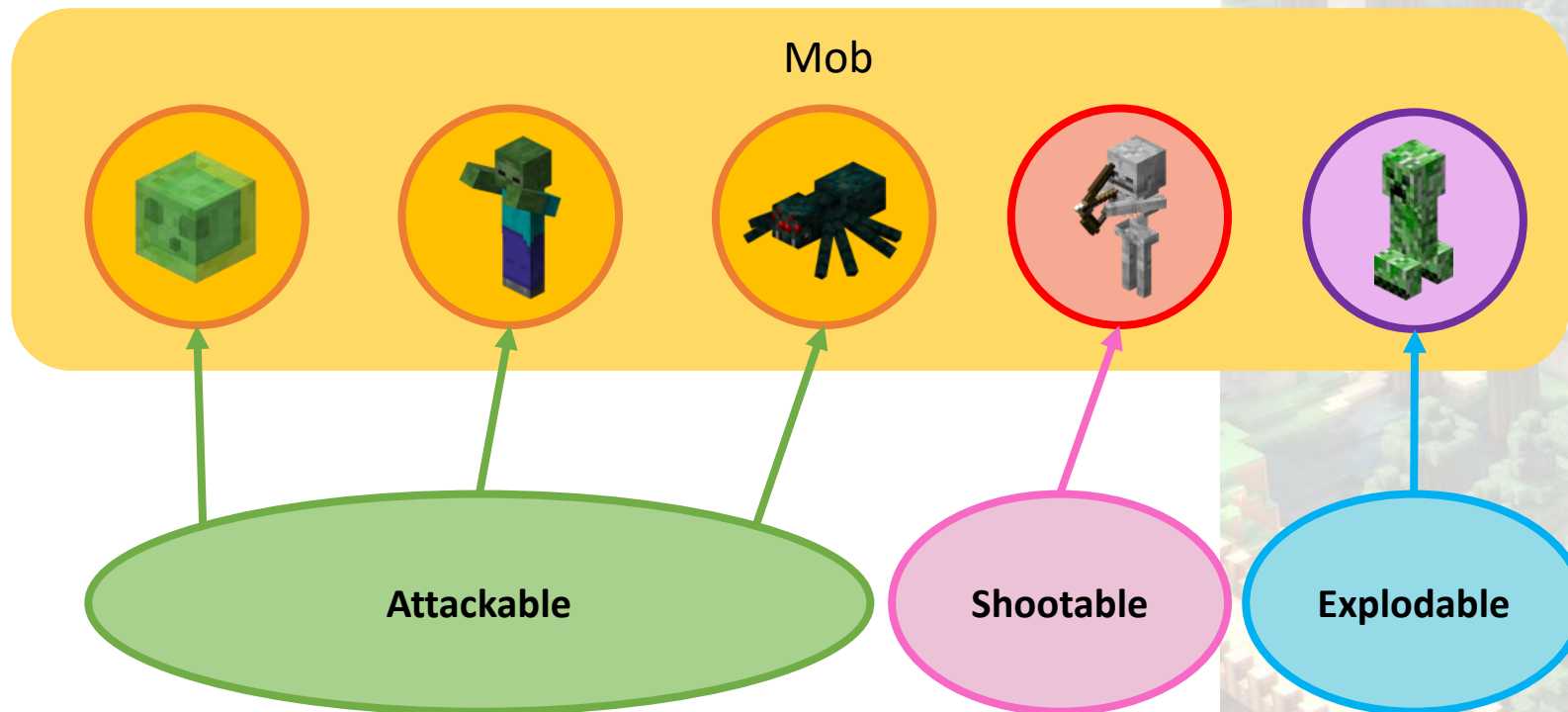


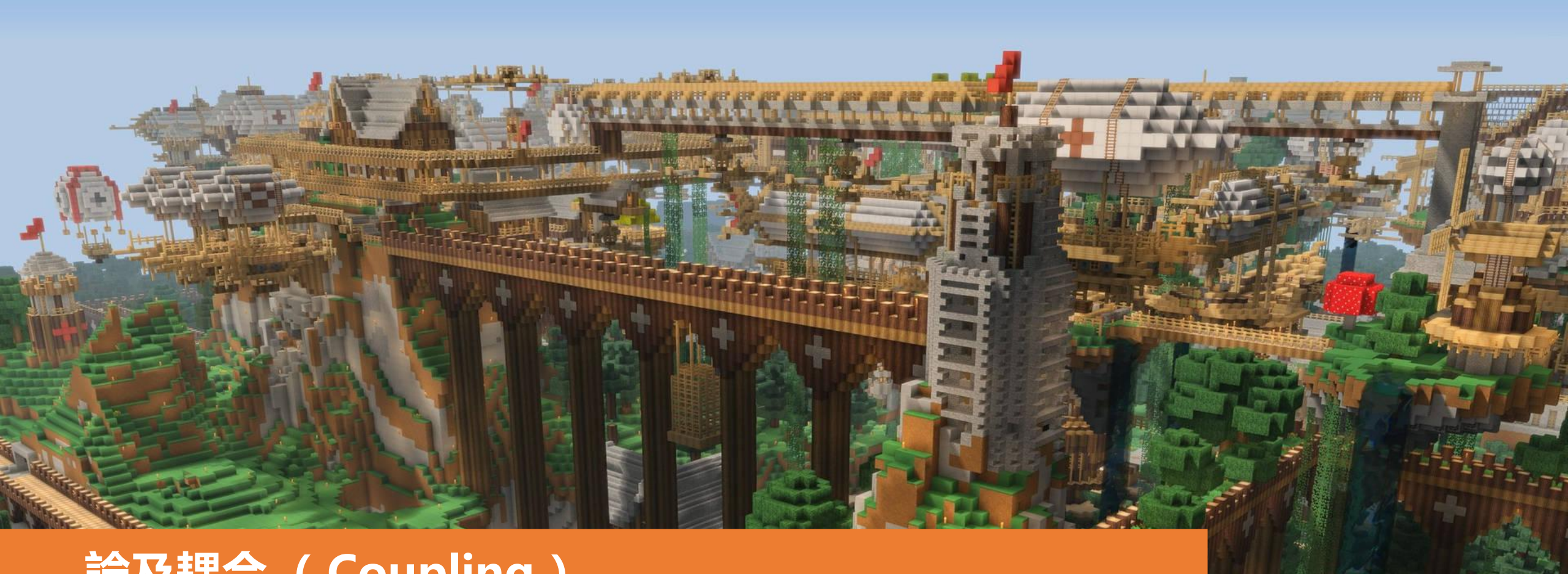
多重繼承：

```
1 // 多重繼承語法
2 Class Spider : public Mob, public Attackable {
3
4 };
```


定義物件導向中的耦合 (Coupling)

- 如果我們今天想要額外加一隻怪物叫做 Creeper，我似乎不用額外變更其他的類別。
- 這樣似乎能夠解決問題？





論及耦合（Coupling）

為什麼我們很在乎耦合這件事情

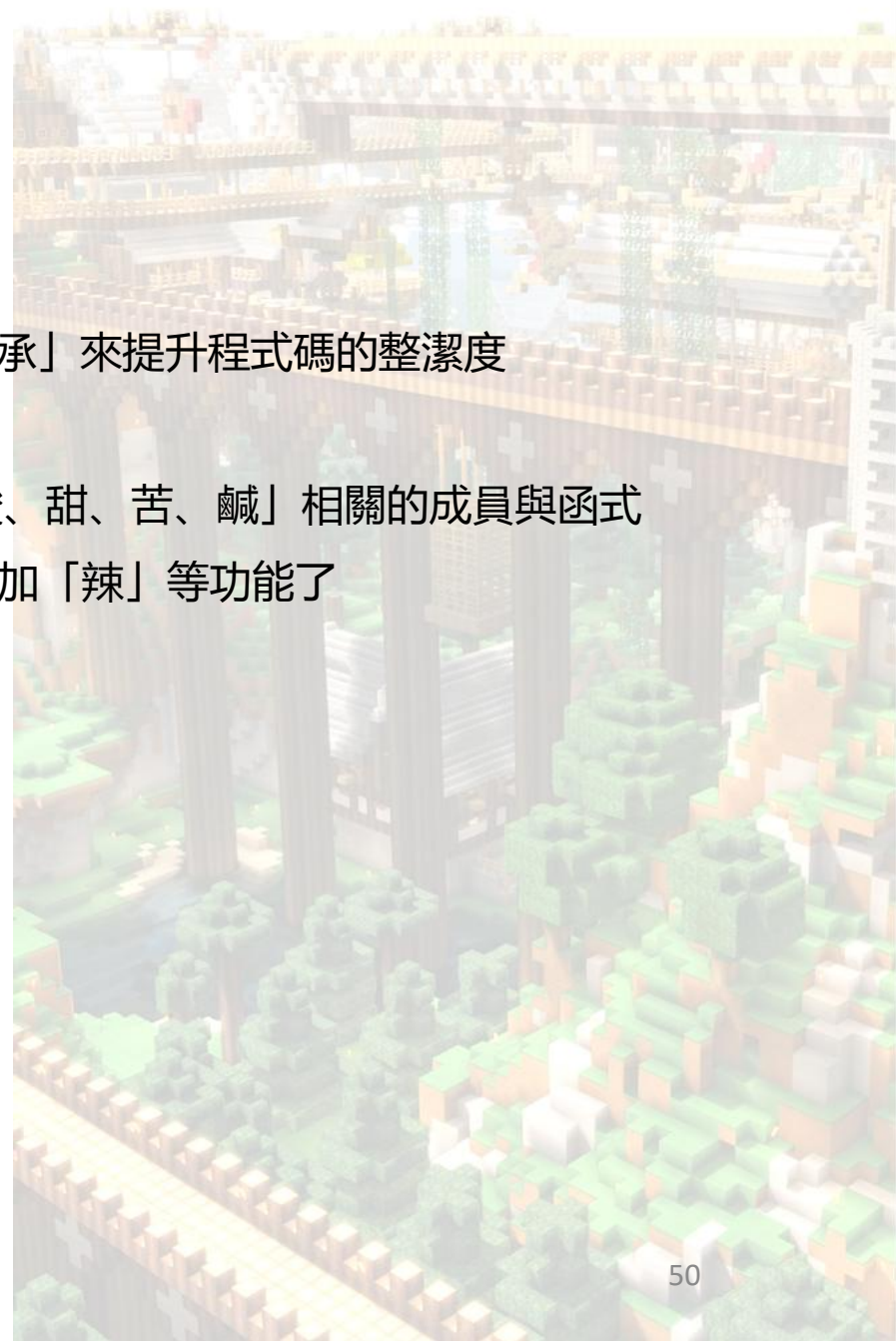
論及耦合 (Coupling)

- 為什麼耦合很重要？
 - 耦合可以協助我們判斷目前這個架構在新增功能 (Features) 時，會不會發生大量變化
- 低耦合 vs. 程式碼整潔度
 - 是一種 Trade off
 - Ex. 大量使用繼承，又能夠保持程式碼整潔度？



論及耦合 (Coupling)

- 這時候，我們可以先思考「這樣的一個東西會不會有大量變動的可能」
 - 如果某些事物，我們可以確保它不會再有額外變動的可能，就能夠使用「繼承」來提升程式碼的整潔度
 - Ex. 我們知道蛋糕是酸、甜、苦、鹹的
 - 我們可以讓所有不同種類的蛋糕，繼承「蛋糕」這個類別，並有與「酸、甜、苦、鹹」相關的成員與函式
 - 理論上，蛋糕不會有辣的（假設☺），我們不會在蛋糕類別上，再額外加「辣」等功能了
 - 這時候，用「繼承」就不需要擔心因為耦合所以導致大量變更的可能
- 所以，我們在乎「耦合」這件事情，主要還是在於**要頻繁處理繼承鏈的變更**
 - 如果這個繼承鏈沒有太大的變動，那其實就可以不用太在意耦合



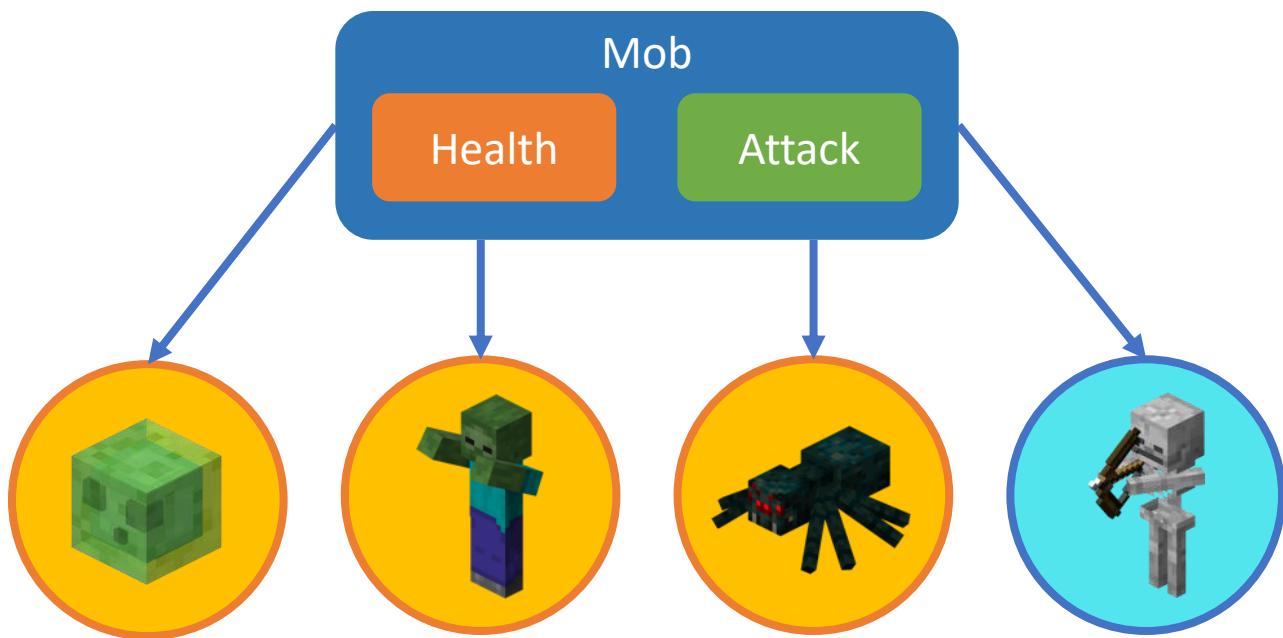


利用「組合與介面」取代繼承「類別」

一種新的程式設計方式，用來取代繼承

傳統類別的繼承

- 當我們使用繼承的時候，我們主要將**重複的程式碼**提到了父類
 - 好處：程式碼的整潔度提升
 - 壞處：使耦合度提升，可能會產生巨大的繼承鏈來導致難以維護

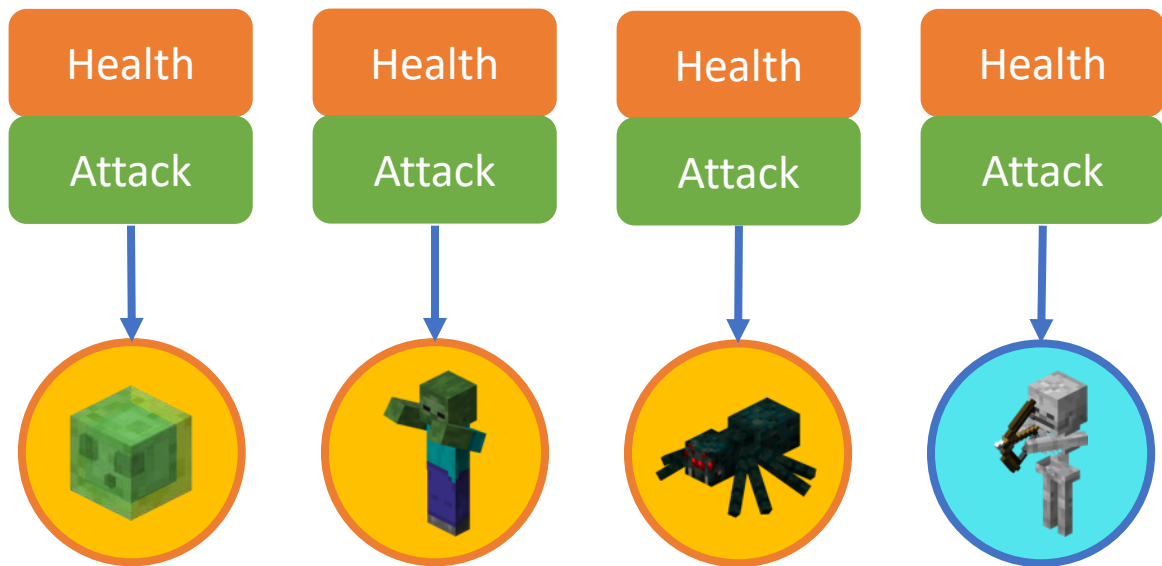


```
class Mob {
private:
    int health;
    int attack;
public:
    Mob() = default;
    Mob(int health, int attack){
        this->health = health;
        this->attack = attack;
    }
    int GetHealth(){
        return health;
    }
    int GetAttack(){
        return attack;
    }
};

class Slime : public Mob {
private:
public:
    Slime(int health, int attack) : Mob(health, attack){
    }
    void Attack(Character *character) {
        character->SetHealth(character->GetHealth() - GetAttack());
    }
    void Split(){
        /* 史萊姆被殺死之後會分裂 */
    }
};
```


一個新方向實現繼承

- 能否使用下面的架構來實作出這種繼承



- 概念有點像是這樣：
 - 不繼承 Mob，而是直接把 Mob 當作類別成員
 - 創立 MobHealth 與 MobAttack 來當作 Mob 內部的成員

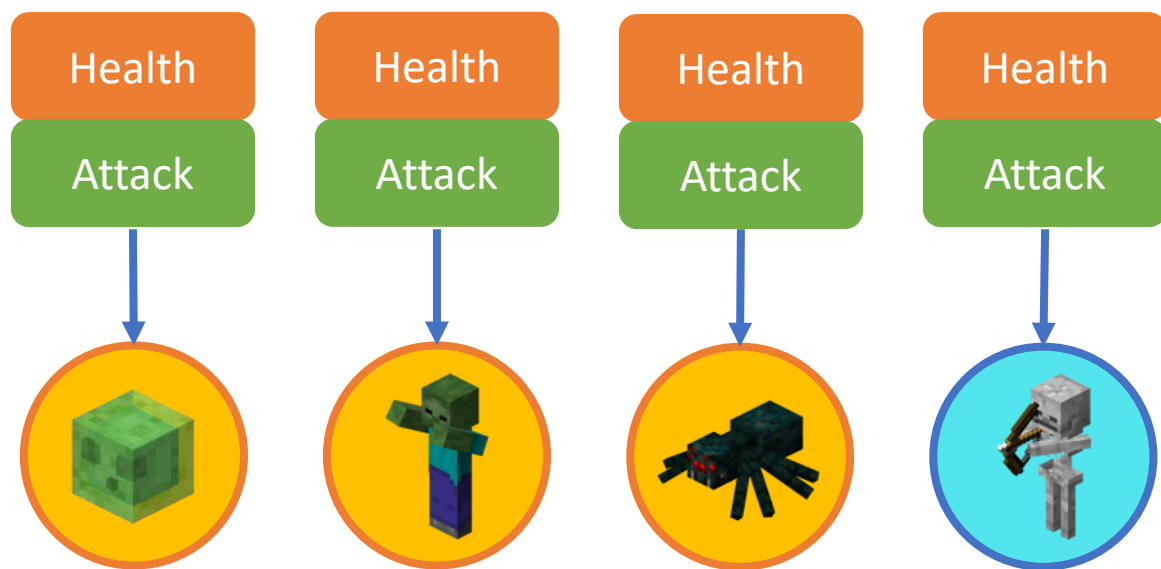
```
class MobHealth {
private:
    int health;
public:
    MobHealth(int health){
        this->health = health;
    }
    int GetHealth(){
        return health;
    }
};

class MobAttack {
private:
    int attack;
public:
    MobAttack(int attack){
        this->attack = attack;
    }
    int GetAttack(){
        return attack;
    }
};

class Slime {
private:
    MobHealth health;
    MobAttack attack;
public:
    Slime(int health, int attack) : health(health), attack(attack){
    }
    void Attack(Character *character) {
        character->SetHealth(character->GetHealth() - attack.GetAttack());
    }
    void Split(){
        /* 史萊姆被殺死之後會分裂 */
    }
};
```

優勢

- 當我們初始化了這些類別後，我們依然能夠擁有 Mob 的特性
- 如果你想要什麼特性，直接拿就行



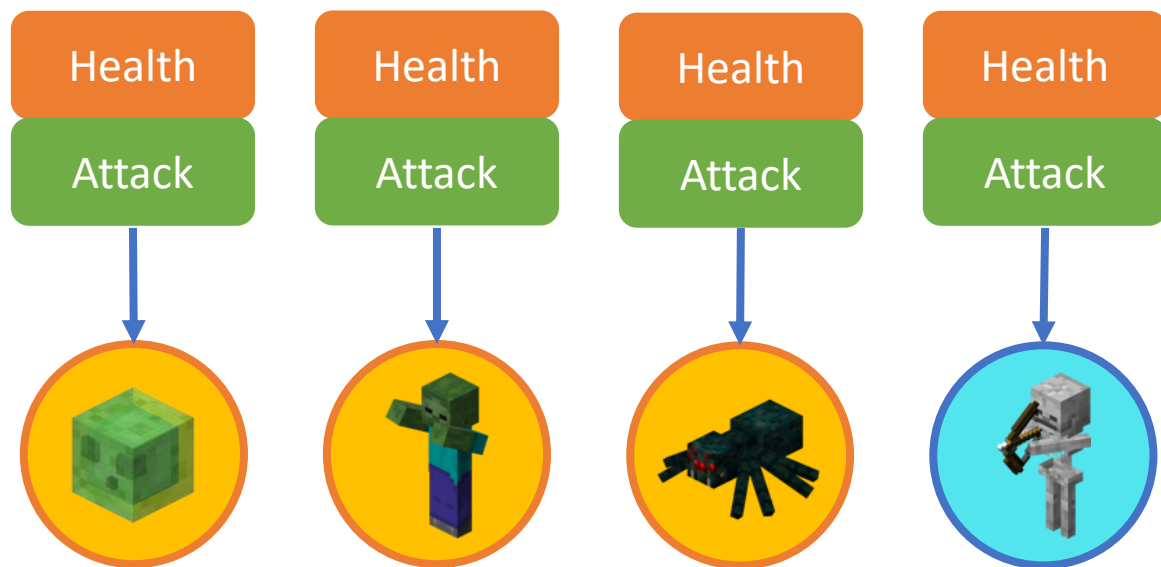
```
class MobHealth {
private:
    int health;
public:
    MobHealth(int health){
        this->health = health;
    }
    int GetHealth(){
        return health;
    }
};

class MobAttack {
private:
    int attack;
public:
    MobAttack(int attack){
        this->attack = attack;
    }
    int GetAttack(){
        return attack;
    }
};

class Slime {
private:
    MobHealth health;
    MobAttack attack;
public:
    Slime(int health, int attack) : health(health), attack(attack){
    }
    void Attack(Character *character) {
        character->SetHealth(character->GetHealth() - attack.GetAttack());
    }
    void Split(){
        /* 史萊姆被殺死之後會分裂 */
    }
};
```

存在問題

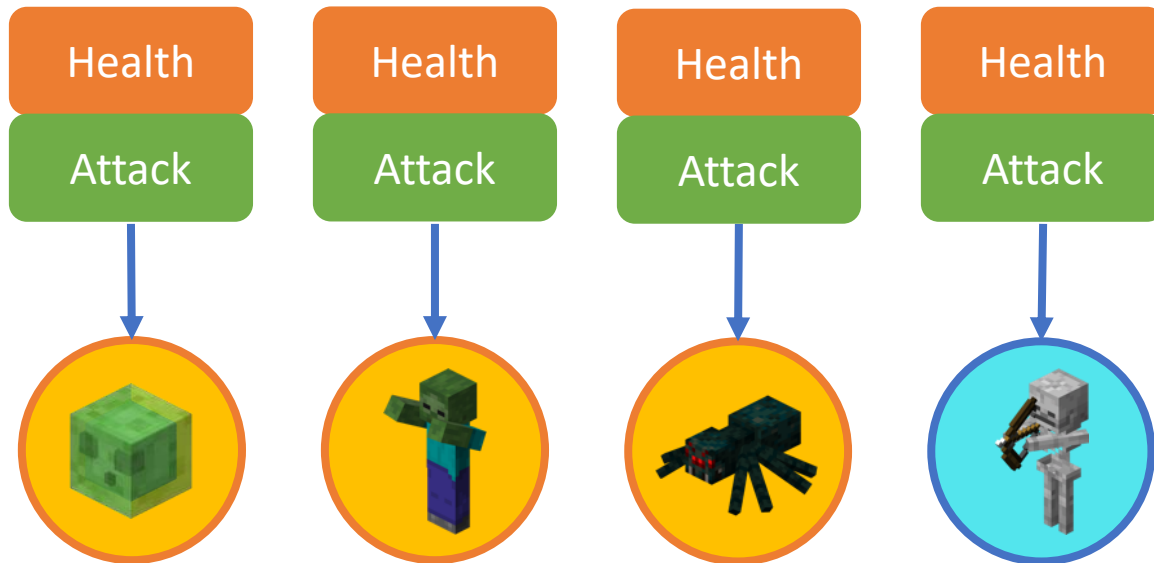
- 成為了另一種設計方法，來讓我們能夠達成繼承，然而存在問題：
 - 我們無法初始化一個有 Pure Virtual Function 的類別的，很顯然的有 Pure Virtual Function 的 Class 是沒辦法被實例化的
 - Ex. 左圖，若 Mob 中有 Pure Virtual Function，無法編譯



```
class Mob {  
private:  
    int health;  
    int attack;  
public:  
    Mob() = default;  
    Mob(int health, int attack){  
        this->health = health;  
        this->attack = attack;  
    }  
    int GetHealth(){  
        return health;  
    }  
    int GetAttack(){  
        return attack;  
    }  
    virtual void Attack(Character *character) = 0;  
};  
  
class Slime {  
private:  
    Mob mob;  
public:  
    Slime(int health, int attack): mob(health, attack) {}  
};
```


一個搞笑的解決方式

- 讓它不要是 Pure
 - 那這樣就喪失了一定要強制定義的意義
- 如果我們喪失了 Virtual Function，就沒辦法做多型這件事情
 - 畢竟 Mob 可以透過 Object Casting 來讓子類都有 Attack 的 Function
 - 但如果我們不這麼做就沒有了多型



```
class Mob {
private:
    int health;
    int attack;
public:
    Mob() = default;
    Mob(int health, int attack){
        this->health = health;
        this->attack = attack;
    }
    int GetHealth(){
        return health;
    }
    int GetAttack(){
        return attack;
    }
    virtual void Attack(Character *character) = 0;
};

class Slime {
private:
    Mob mob;
public:
    Slime(int health, int attack){
        mob = Mob(health, attack);
    }
    void Attack(Character *character){
        character->Attack(mob);
    }
    void Split(){
        /* 史萊姆分裂成兩個 */
    }
};
```

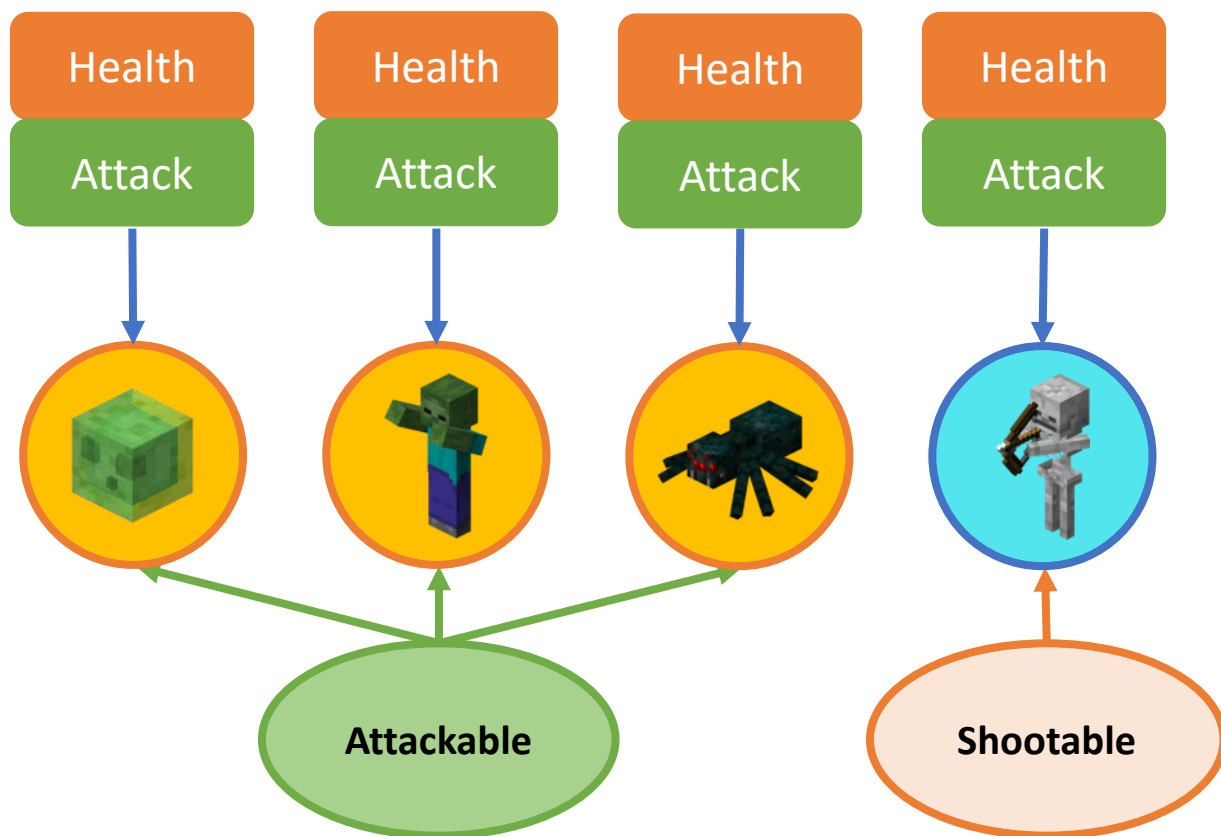
Field type 'Mob' is an abstract class
unimplemented pure virtual method 'Attack' in 'Mob'

Declared in: Practice_1b.cpp

private:
Mob Slime::mob

利用組合與介面取代繼承類別

- 這時候我們可以使用介面，來讓所有的類別繼承介面。
- 由於介面是由 Pure Virtual Function 組成，所以就會強制實作。



```
class MobHealth {
private:
    int health;
public:
    MobHealth(int health){
        this->health = health;
    }
    int GetHealth(){
        return health;
    }
};

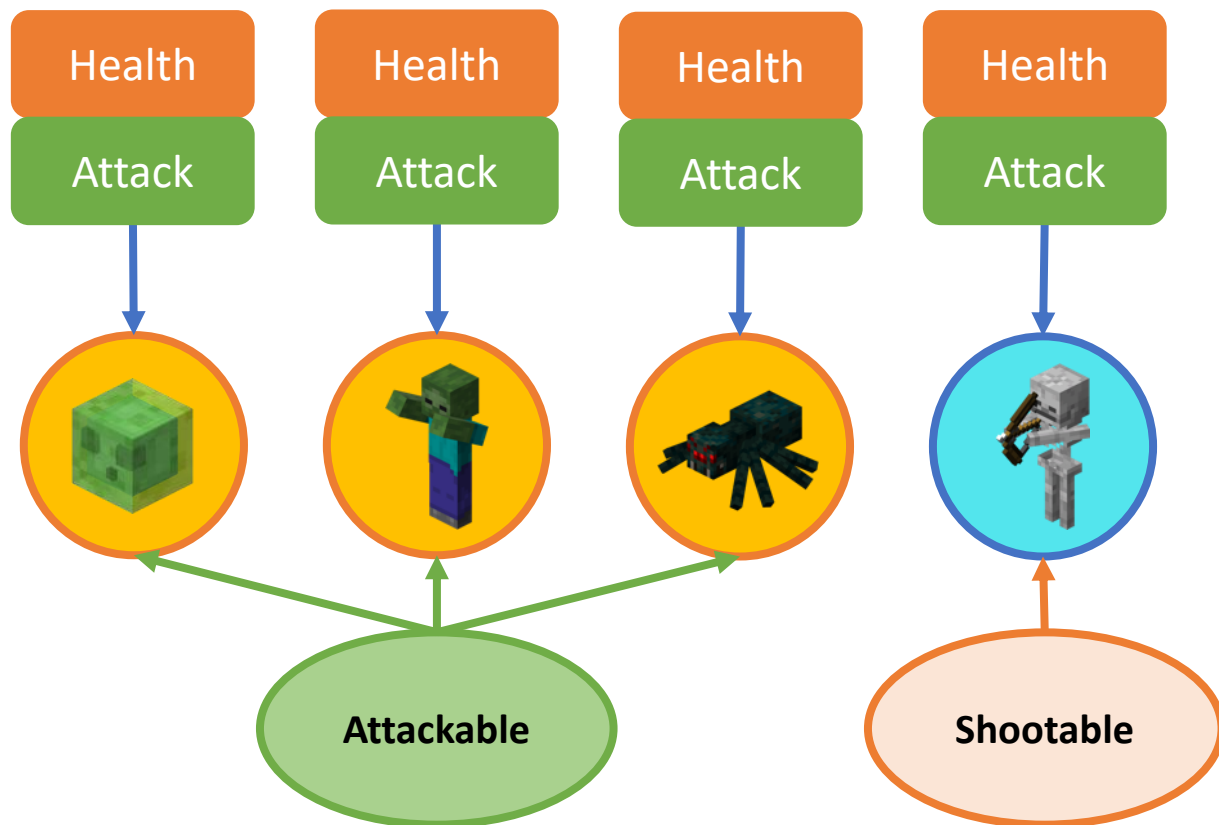
class MobAttack {
private:
    int attack;
public:
    MobAttack(int attack){
        this->attack = attack;
    }
    int GetAttack(){
        return attack;
    }
};

class IAttackable {
public:
    virtual void Attack(Character *character) = 0;
};

class Slime : public IAttackable {
private:
    MobHealth health;
    MobAttack attack;
public:
    Slime(int health, int attack) : health(health), attack(attack){
    }
    void Attack(Character *character) override {
        character->SetHealth(character->GetHealth() - attack.GetAttack());
    }
    void Split(){
        /* 史萊姆被殺死之後會分裂 */
    }
};
```

利用組合與介面取代繼承類別

- 透過這樣的方式，我們可以取代繼承類別，改用組合與介面來完成。



```
class MobHealth {
private:
    int health;
public:
    MobHealth(int health){
        this->health = health;
    }
    int GetHealth(){
        return health;
    }
};

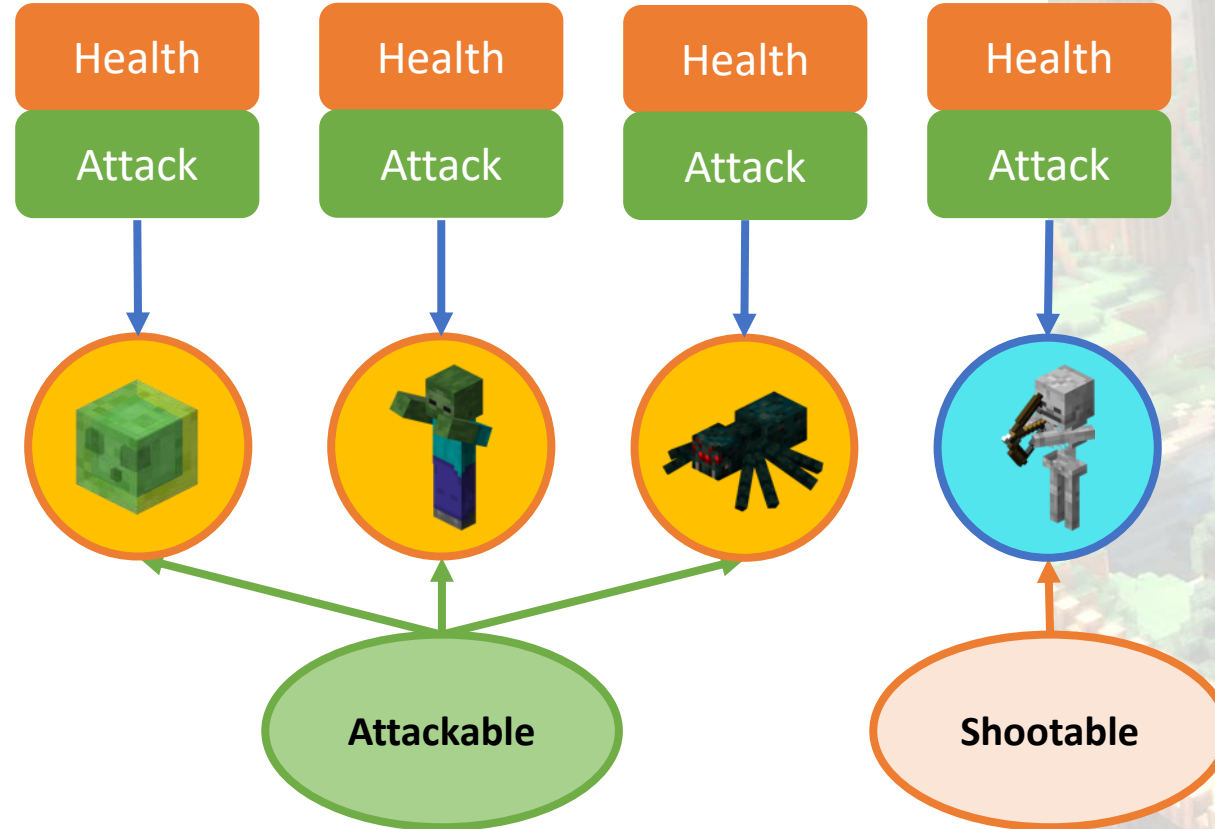
class MobAttack {
private:
    int attack;
public:
    MobAttack(int attack){
        this->attack = attack;
    }
    int GetAttack(){
        return attack;
    }
};

class IAttackable {
public:
    virtual void Attack(Character *character) = 0;
};

class Slime : public IAttackable {
private:
    MobHealth health;
    MobAttack attack;
public:
    Slime(int health, int attack) : health(health), attack(attack){
    }
    void Attack(Character *character) override {
        character->SetHealth(character->GetHealth() - attack.GetAttack());
    }
    void Split(){
        /* 史萊姆被殺死之後會分裂 */
    }
};
```

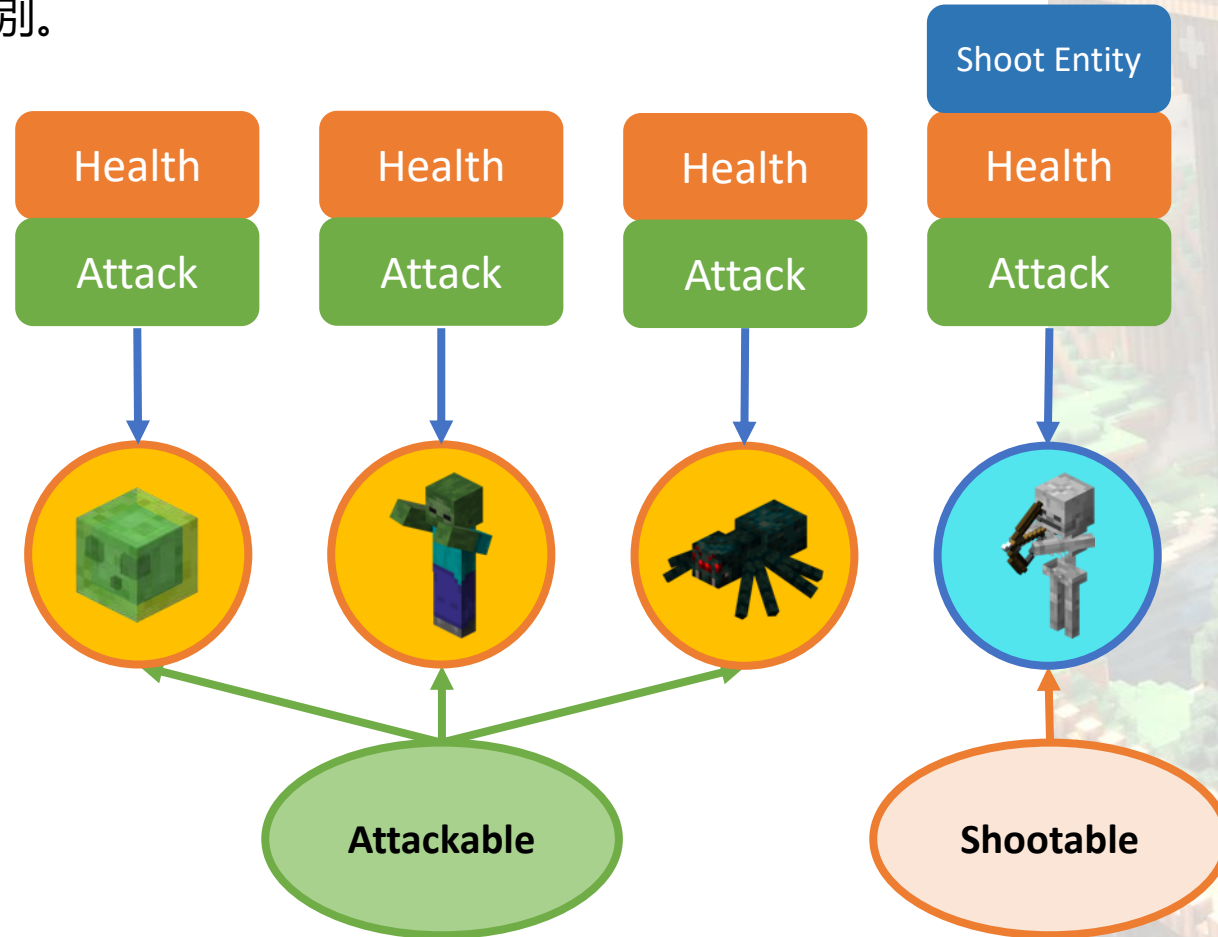

利用組合與介面取代繼承類別

- 這樣做有什麼好處？
- 我們實際上是在「扁平化」繼承鏈，如果繼承鏈的高度變低，那麼就會讓耦合度下降，可能使程式碼更好維護。



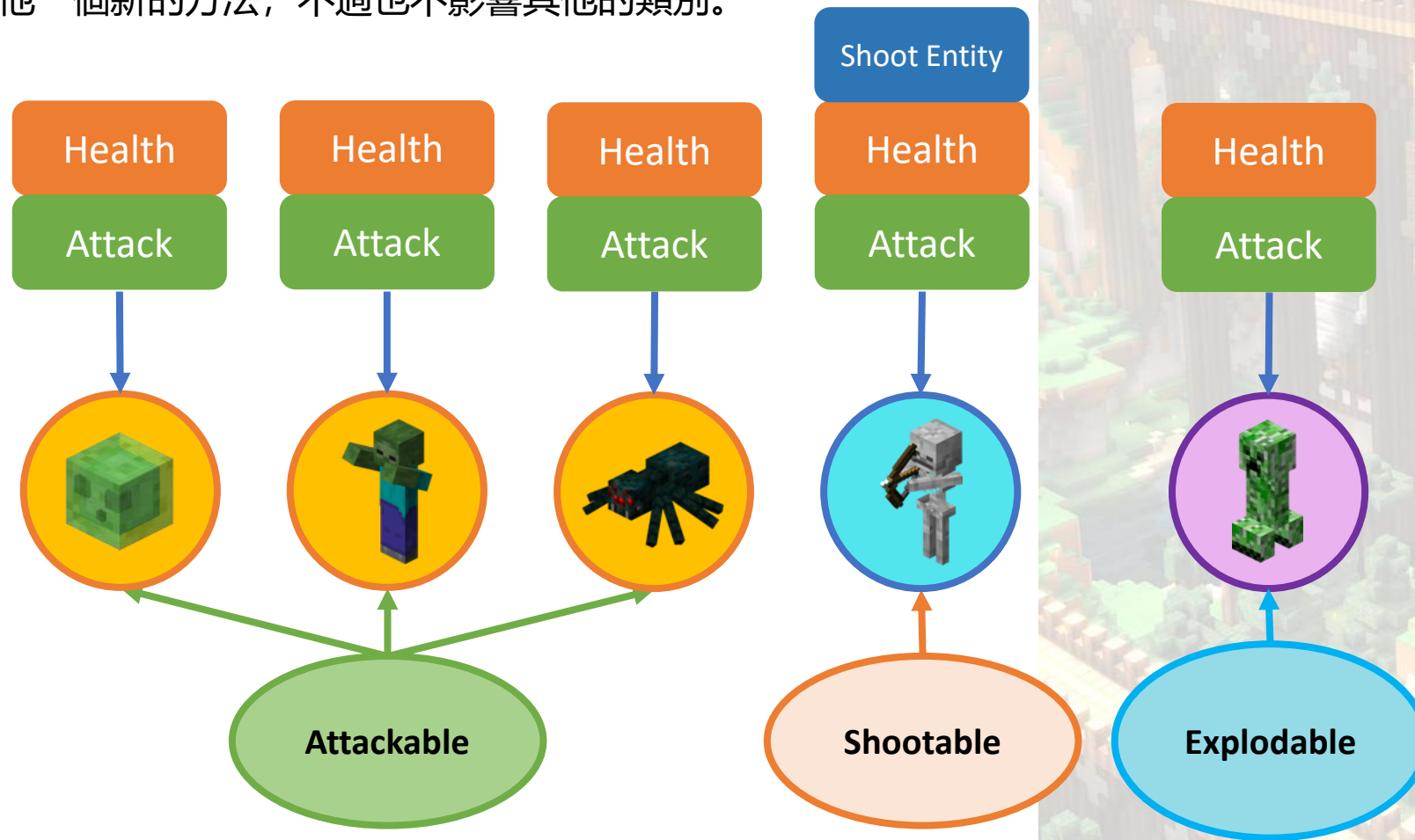
利用組合與介面取代繼承類別

- 這些 Class 都在同一層上，如果我們希望可以對一個特定的類別新增一個類別成員...
- 那似乎也不影響其他的類別。



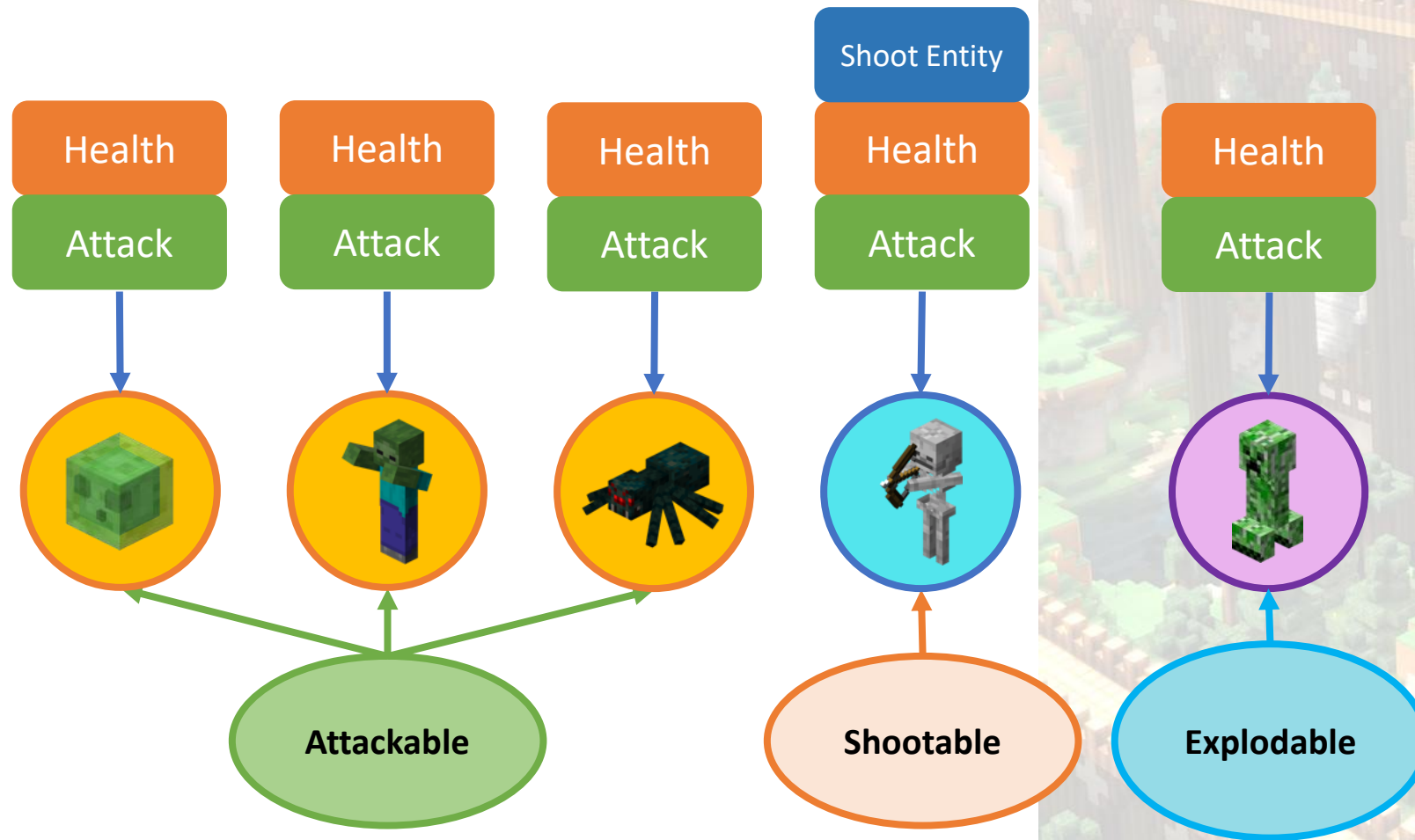
利用組合與介面取代繼承類別

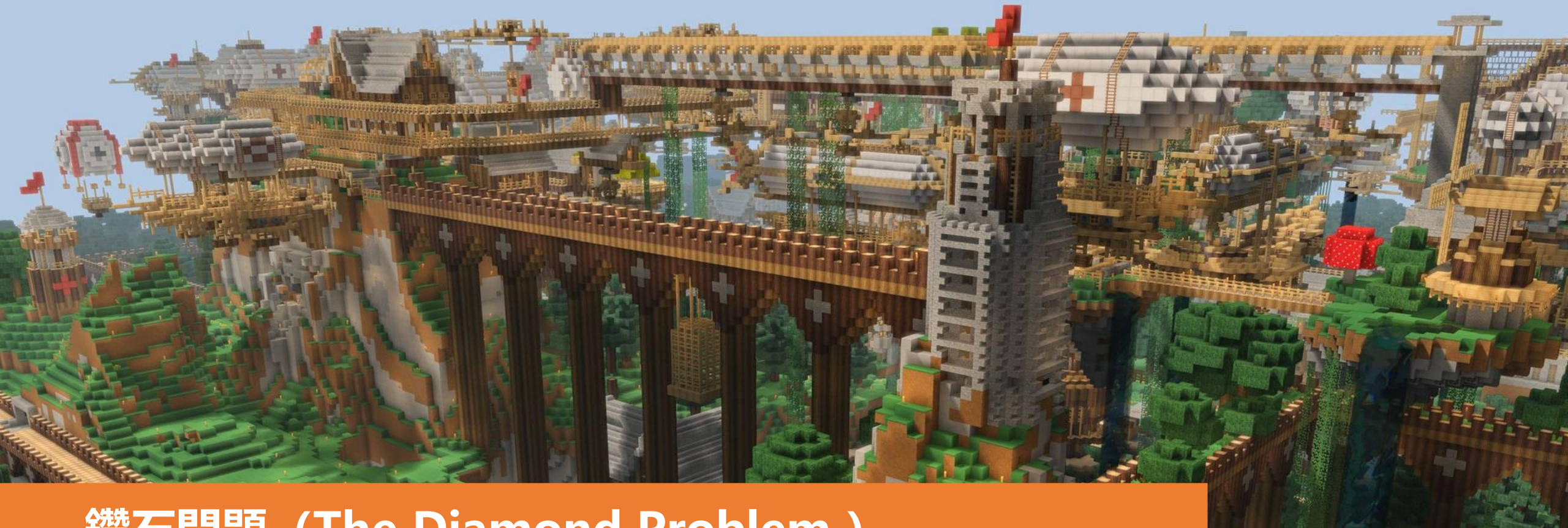
- 又或者是如果我們希望能夠有一個會爆炸的怪物，他攻擊人的方式既不是近戰，也不是射擊，而是爆炸。
- 那麼我們可能要給他一個新的方法，不過也不影響其他的類別。



利用組合與介面取代繼承類別

- 透過這樣的低耦合度實作，可以讓我們在維護一個可能變動會很大的類別時，能夠影響較少的類別並成功新增功能。



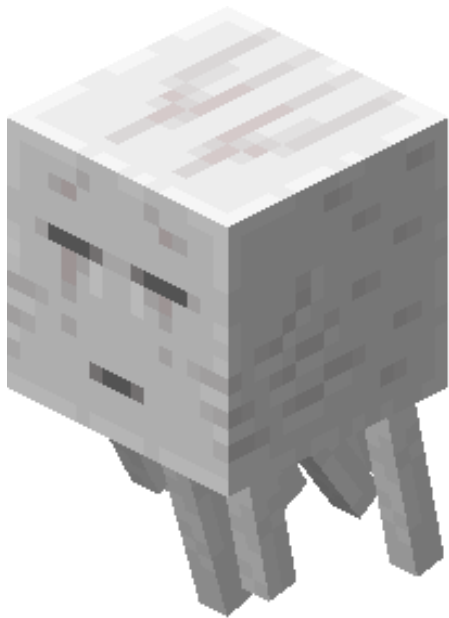


鑽石問題 (The Diamond Problem)

透過介面來解決掉鑽石問題

The Diamond Problem

- 在 Minecraft 中有一種怪物叫做「地獄幽靈」
 - 它會飄浮在空中 (Floatable)
 - 並會對玩家發射火球 (Shootable)

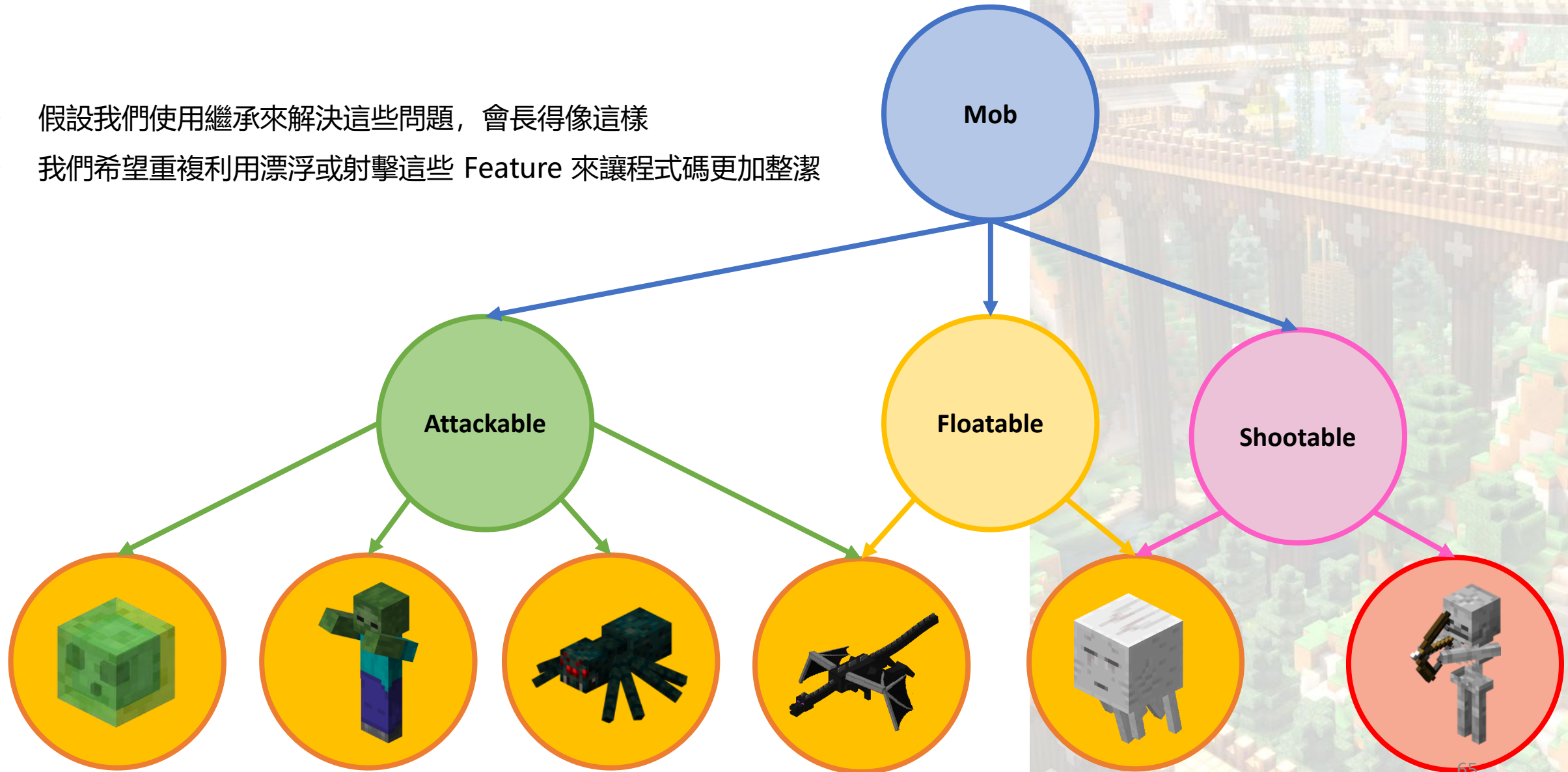


- 另外「自製終界龍」也會飛，同時會以時速 200km 衝撞玩家，造成玩家角色直接死亡
 - 它會飄浮在空中 (Floatable)
 - ~~• 對玩家發射火球 (Shootable)~~
 - 並會有近戰攻擊 (Attackable)

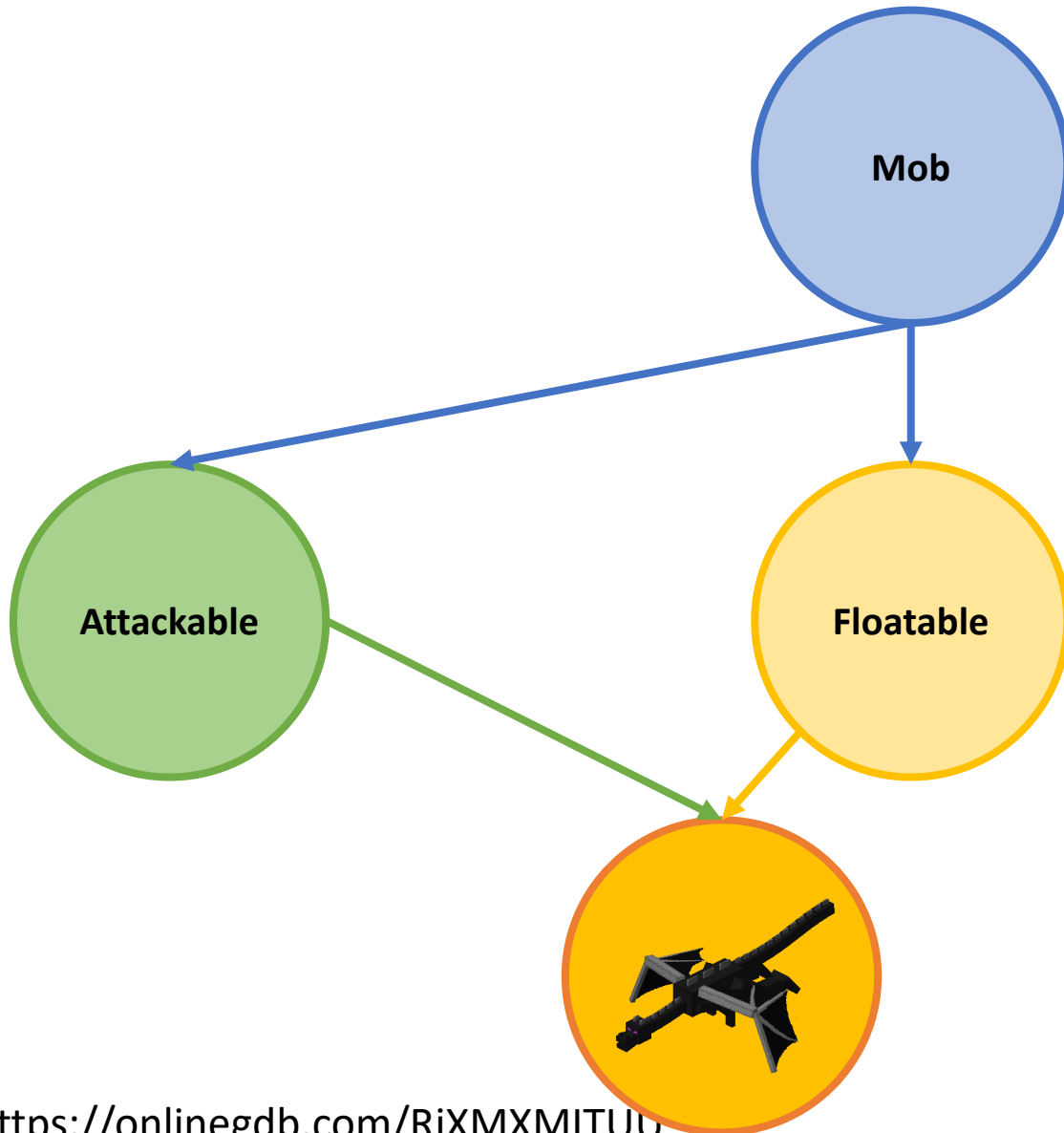


The Diamond Problem

- 假設我們使用繼承來解決這些問題，會長得像這樣
- 我們希望重複利用漂浮或射擊這些 Feature 來讓程式碼更加整潔



The Diamond Problem

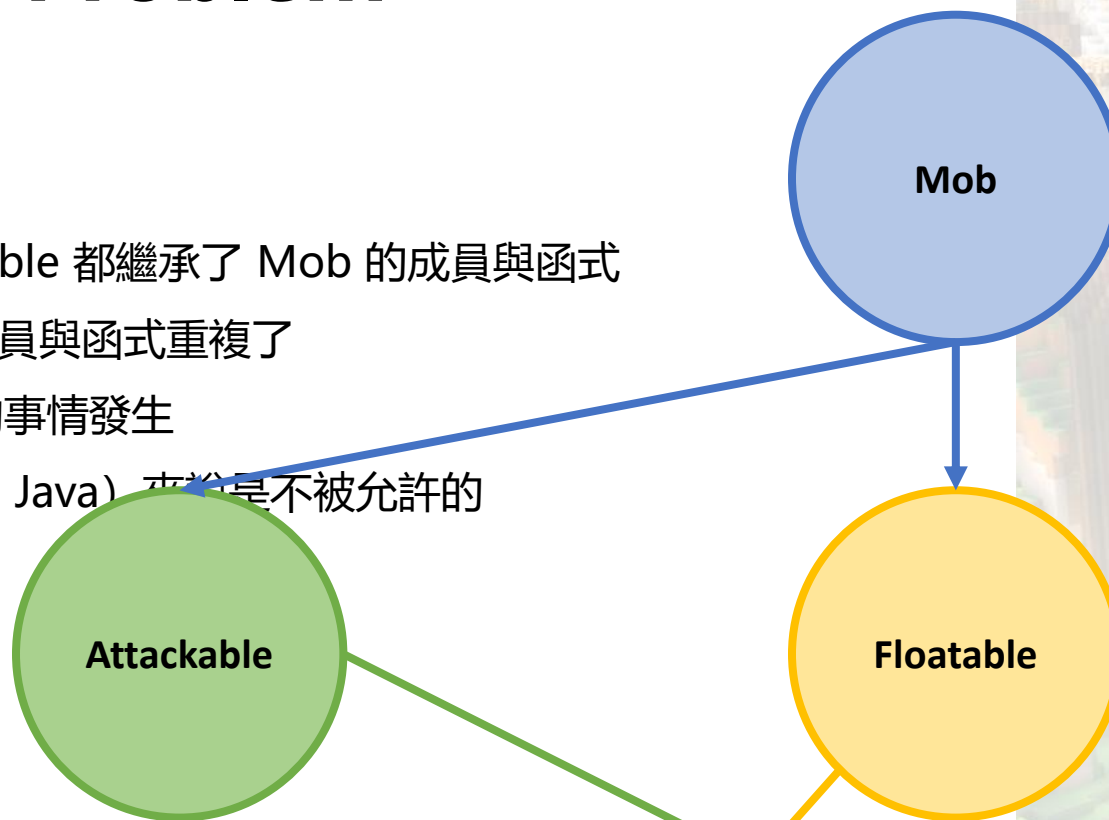


```
3 class Mob {
4 public:
5     virtual void action() = 0;
6 };
7
8 class Attackable : public Mob {
9 public:
10     void action() override {
11         std::cout << "Attack!" << std::endl;
12     }
13 };
14
15 class Floatable : public Mob {
16 public:
17     void action() override {
18         std::cout << "Float!" << std::endl;
19     }
20 };
21
22 class EnderDragon : public Attackable, public Floatable {
23     // 不覆寫action(), 導致歧義
24 };
25
26 int main() {
27     EnderDragon enderdragon;
28     enderdragon.action(); // 這行將會引發編譯錯誤
29     return 0;
30 }
```



The Diamond Problem

- 以「自製終界龍」來舉例
 - Attackable 與 Floatable 都繼承了 Mob 的成員與函式
 - 衝突發生! Mob 的成員與函式重複了
- 多重繼承下, 會導致這樣的事情發生
 - 對於一些語言 (例如: Java) 來說是不被允許的
 - 在編譯期間就會擋掉

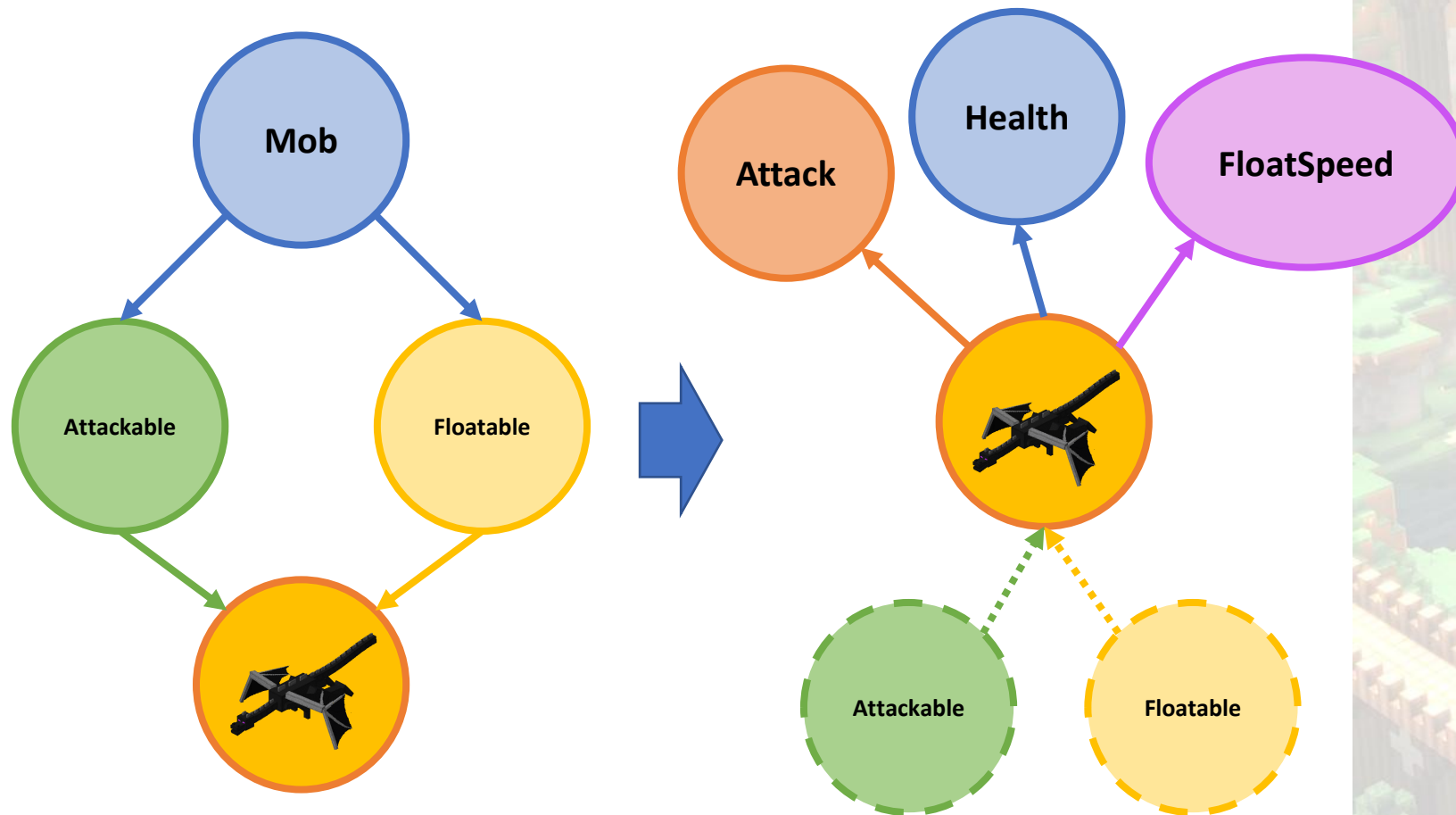


- 那這樣, 我們就不能讓這條龍會飛又會衝撞玩家了嗎?

由於我們會有兩組的 Mob, 所以這個時候如果我們使用了 Mob 的函數, 會產生歧異 (Ambiguous)

解決 Diamond Problem

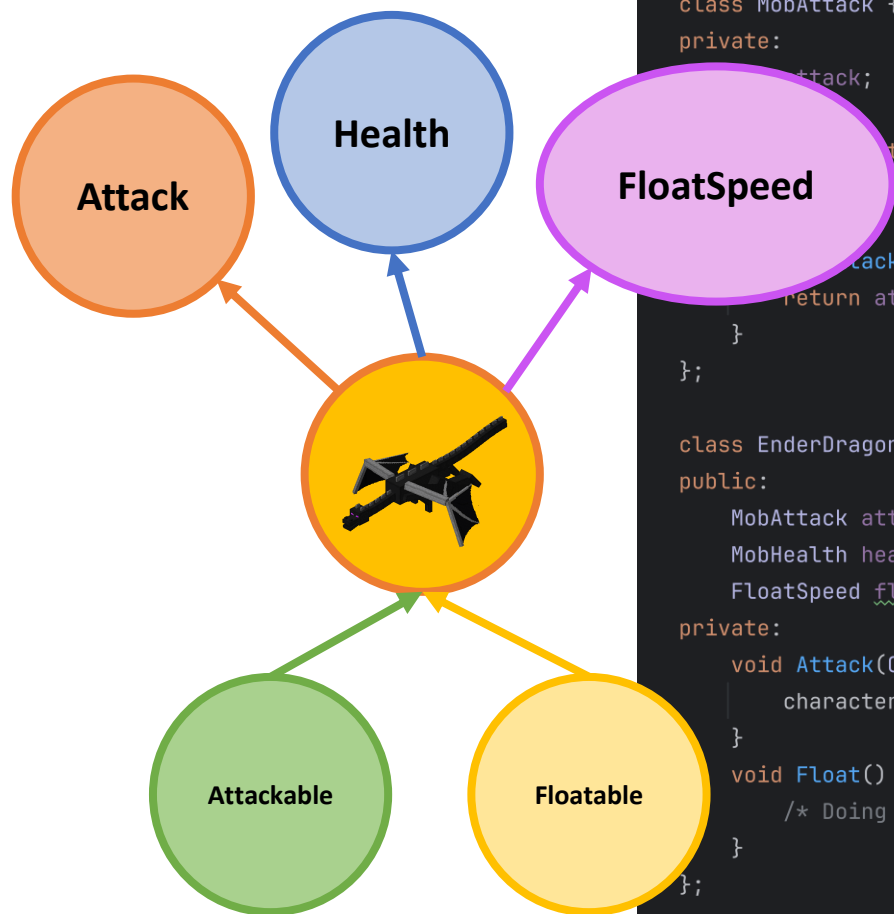
- 透過「組合與介面」來避免 Diamond Problem 發生
 - 扁平化繼承鏈



- 我們透過把 Mob 與 Floatable 中 FloatSpeed 當作成員，並且把近戰攻擊的 Attackable 與漂浮的 Floatable 當作介面。
- 透過這樣的方式，我們可以將繼承鏈從兩層減少至一層，並且能夠解決掉鑽石問題。

Code example

- 實作細節就像這個樣子。



```
class MobHealth {
private:
    int health;
public:
    MobHealth(int health){
        this->health = health;
    }
    int GetHealth(){
        return health;
    }
};

class MobAttack {
private:
    MobAttack * attack;
public:
    MobAttack(MobAttack * attack){
        this->attack = attack;
    }
    MobAttack * GetAttack(){
        return attack;
    }
};

class EnderDragon : public IAttackable, public IFloatable {
public:
    MobAttack attack;
    MobHealth health;
    FloatSpeed floatspeed;
private:
    void Attack(Character *character) override{
        character->SetHealth(0);
    }
    void Float() override{
        /* Doing some float with floatspeed */
    }
};
```

Code example

- 實作細節

```
15 class MobAttack {
16 private:
17     int attack;
18
19 public:
20     MobAttack(int attack) {
21         this->attack = attack;
22     }
23
24     int GetAttack() {
25         return attack;
26     }
27 };
```

Attack

```
1 class MobHealth {
2 private:
3     int health;
4
5 public:
6     MobHealth(int health) {
7         this->health = health;
8     }
9
10    int GetHealth() {
11        return health;
12    }
13 };
```

Health

```
29 class FloatSpeed {
30 private:
31     float speed;
32
33 public:
34     FloatSpeed(float speed) {
35         this->speed = speed;
36     }
37
38     float GetSpeed() {
39         return speed;
40     }
41 };
```

FloatSpeed

```
72 EnderDragon : public IAttackable, public IFloatable {
73 private:
74     MobAttack attack;
75     MobHealth health;
76     FloatSpeed floatspeed;
77
78 public:
79     EnderDragon(MobAttack attack, MobHealth health, FloatSpeed floatspeed)
80         : attack(attack), health(health), floatspeed(floatspeed) {}
81
82     // 實作 IAttackable 的 Attack 方法
83     void Attack(Character *character) override {
84         character->SetHealth(0);
85     }
86
87     // 實作 IFloatable 的 Float 方法
88     void Float() override {
89         /* Doing some float with floatspeed */
90     }
91 };
```



Attackable

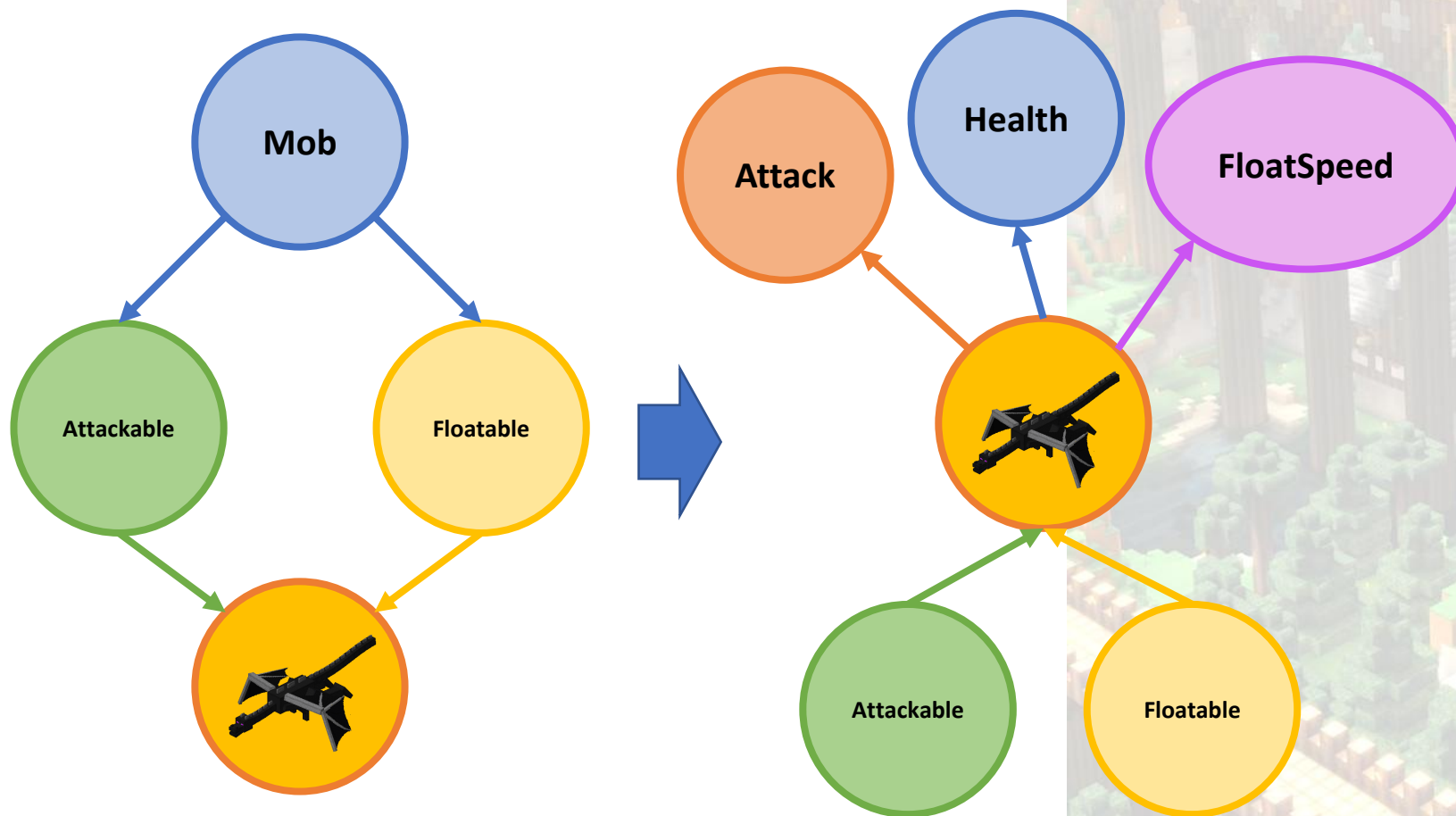
```
43 class IAttackable {
44 public:
45     virtual void Attack(Character *character) = 0;
46 };
```

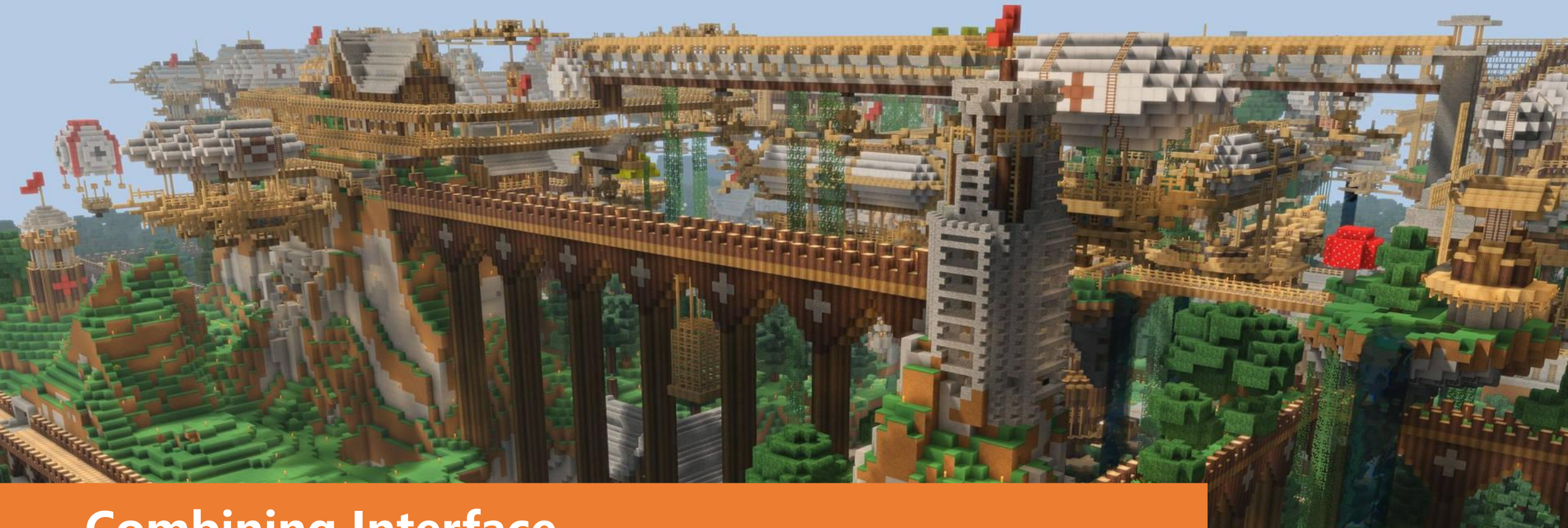
Floatable

```
48 class IFloatable {
49 public:
50     virtual void Float() = 0;
51 };
```


章節結論

- 透過介面與組合，我們可以有效解決鑽石問題所帶來的問題。
- 透過扁平化繼承鏈，使繼承鏈原先可能導致出現鑽石問題，在扁平化後使鑽石變得不存在。



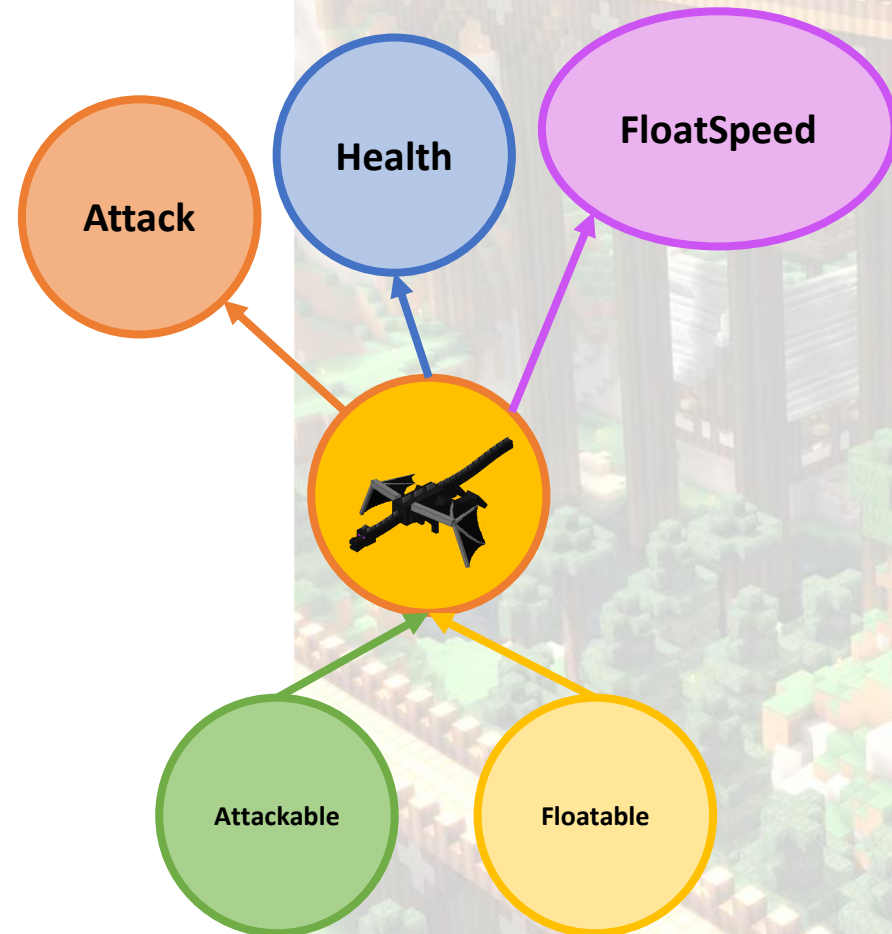
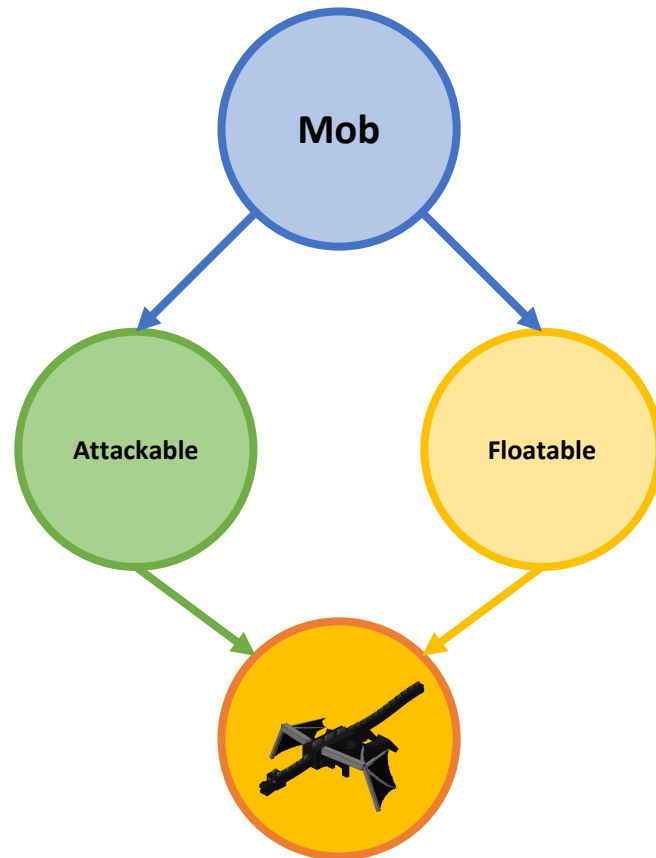


Combining Interface

透過多個介面來組合出一些想要的功能

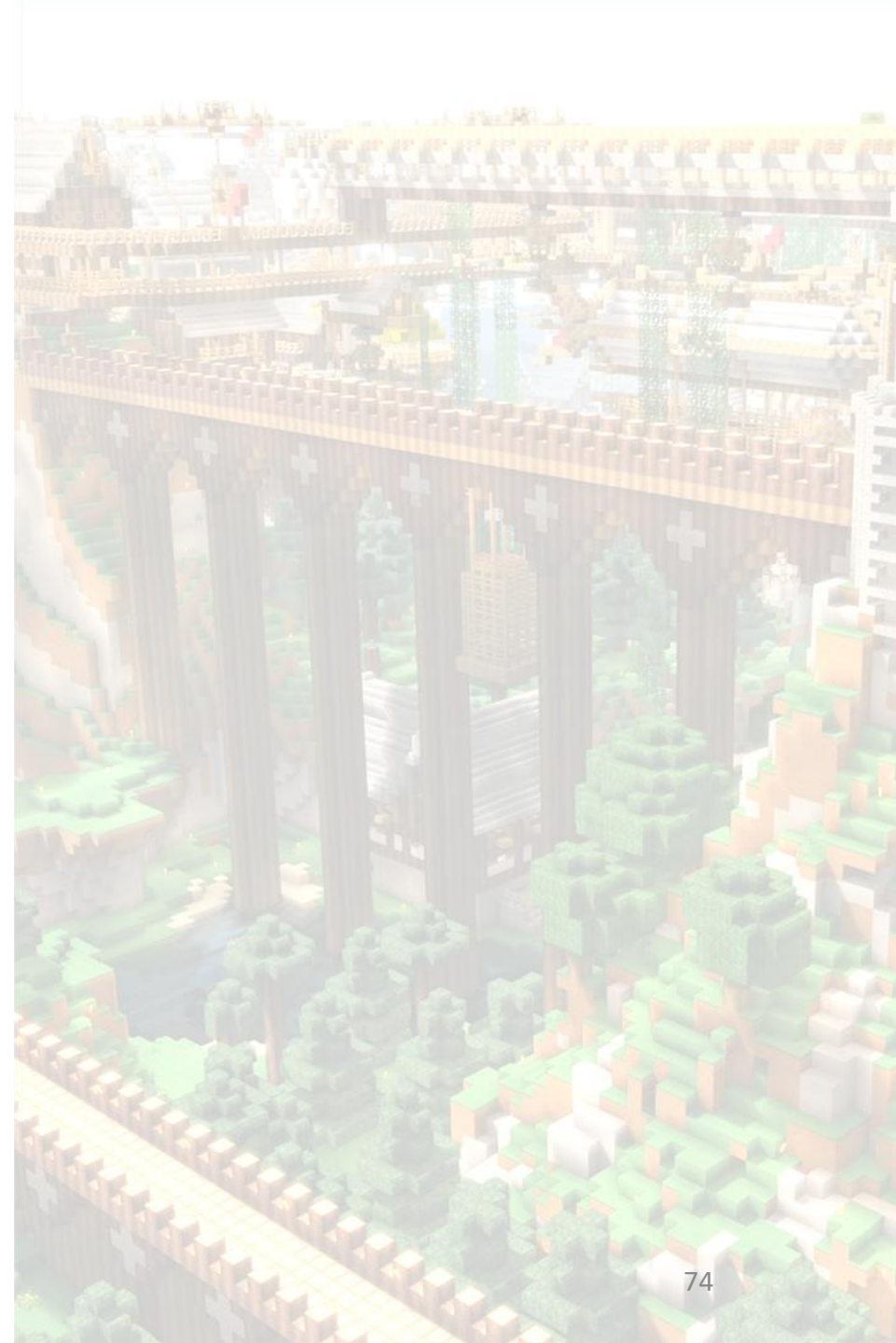
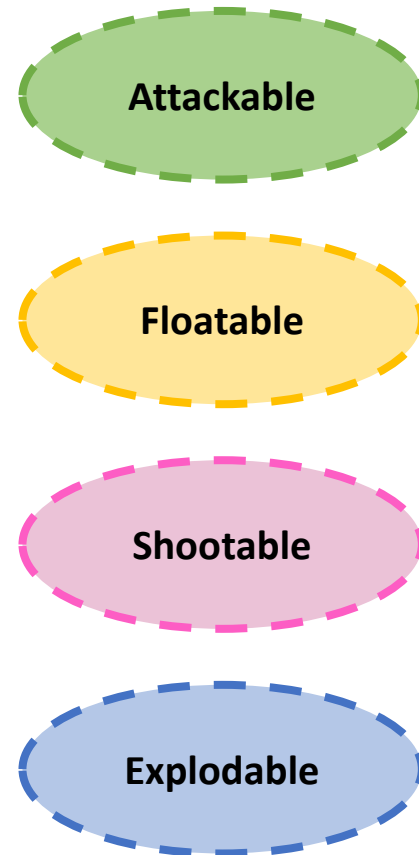
Combining Interface

- 前面有提到扁平化繼承鏈，這個扁平化繼承鏈還有什麼好處呢？



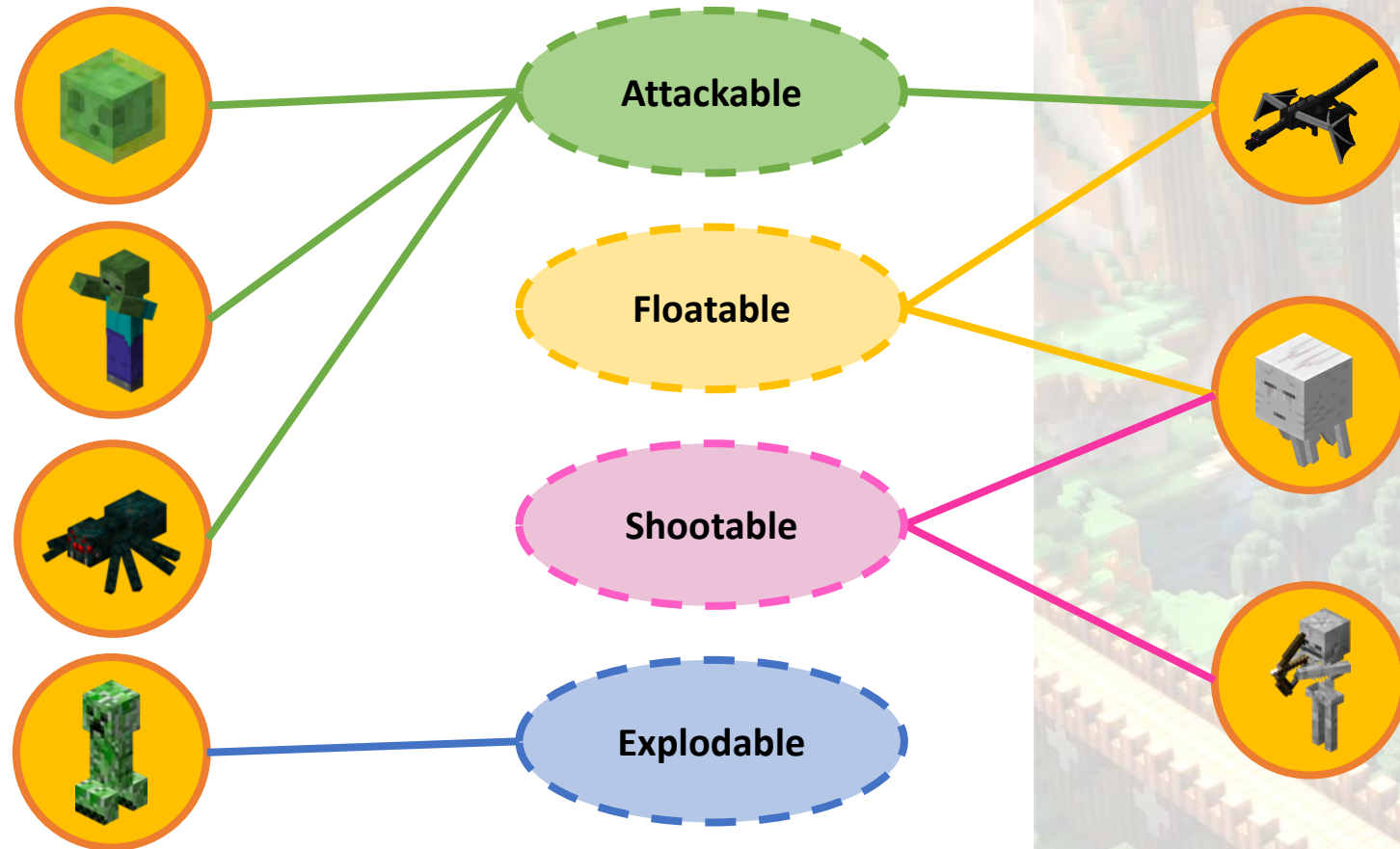
Combining Interface

- 當我們把所有的介面定義好之後，就會像下圖這樣。



Combining Interface

- 把介面定義好的好處就在於，我們可以重複使用這些介面。



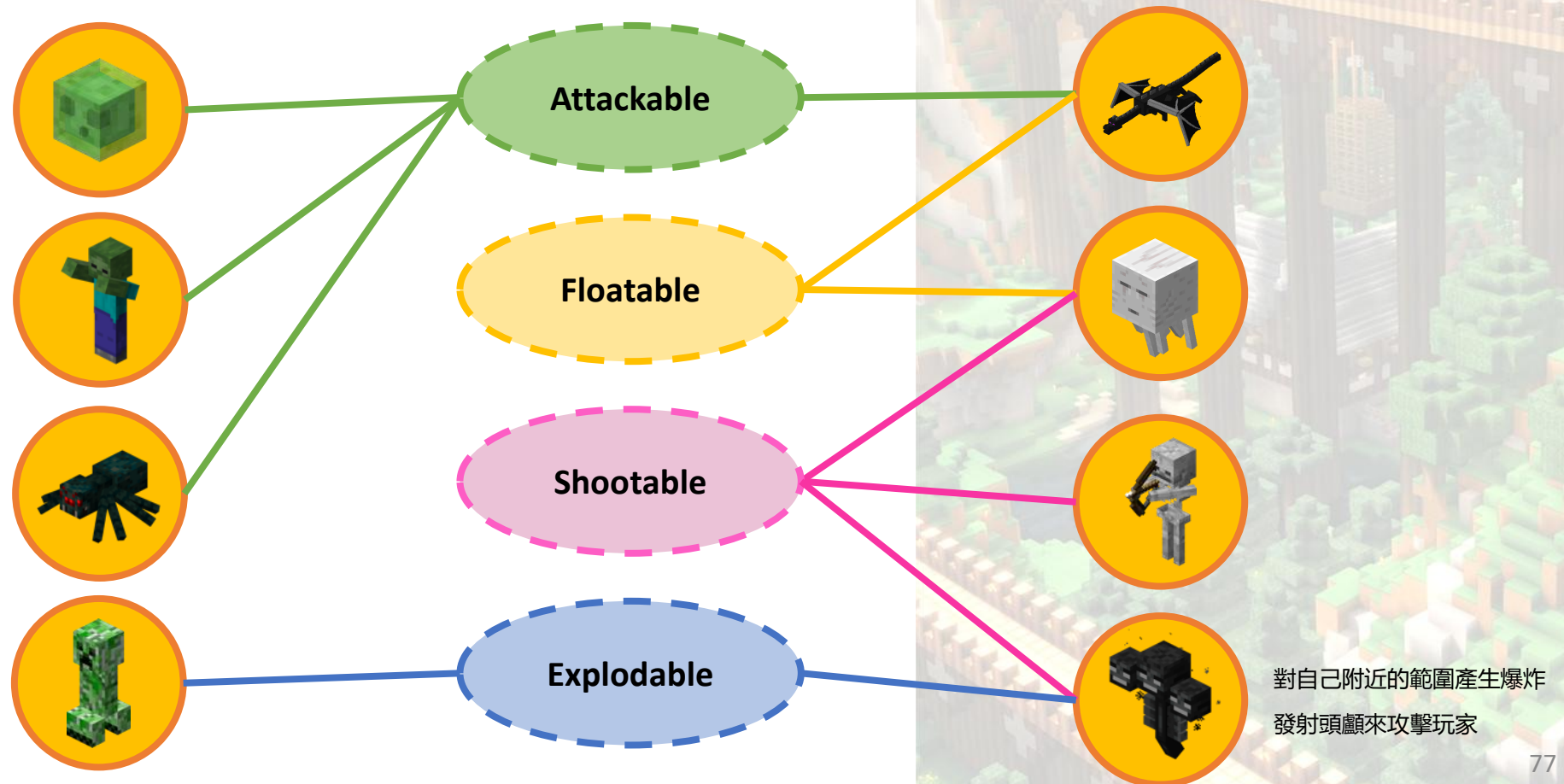
Combining Interface

- 透過 Interface 所帶來的扁平化設計，我們可以隨時透過自己設計的 Interface 來組出想要的怪物屬性
- Ex. 這傢伙叫做「凋零怪」
 - 會在生成時對自己附近的範圍產生爆炸
 - 並且發射頭顱來攻擊玩家



Combining Interface

- 當我們要創立這個怪物時，我們可以直接用介面組成。



Combining Interface

- 那麼我們就能用我們已經組好的介面來創立怪物
 - 使介面能夠重複應用，又能保有扁平化設計的好處
- 假設我們今天又多了一隻怪物...
 - 確認有沒有現有的介面能夠組出怪物的特性
 - **有**：透過介面來組出怪物的特性，又不會影響其他怪物
 - **沒有**：沒關係，我們再透過創建出新介面來有這個特性

介面

```
class IFloatable {  
    virtual void Float() = 0;  
};  
  
class IShootable {  
    virtual void Shoot(Character character) = 0;  
};  
  
class IExplodable {  
    virtual void Explode(int range) = 0;  
};  
  
class EnderDragon : public IShootable, public IFloatable {  
    void Shoot(Character character) override{  
        /* 發射東西給人 */  
    }  
    void Float() override{  
        /* 產生漂浮的行為 */  
    }  
};
```

繼承介面

強制實作function

```
class Wither : public IShootable, public IExplodable {  
    void Shoot(Character character) override{  
        /* 發射東西給人 */  
    }  
    void Explode(int range) override{  
        /* 創造一個大爆炸 */  
    }  
};
```




Template Issue

透過介面來解決掉 Template 最小需求不明確的問題

Template Issue

- 回到之前我們在製作 Template Stack 的例子
- 這個例子告訴我們：
 - 你丟進去的型態必須要支援 Smelt()
 - 只要你丟進去的型態有支援 Smelt(), 那麼你什麼都能丟得進去

```
template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
    template <typename VT>
    VT smelt_many(int n){
        VT container = VT();
        for(int i = 0; i < n; i++){
            container.push_back(smelt());
        }
        return container;
    }
};
```

Template Issue

- 但假設今天的最小需求是很複雜的情況？
- 例如：
 - 你丟進去的型態必須要支援 Smelt()
 - 你丟進去的型態必須要支援 Count()
 - 你丟進去的型態必須要支援 GetSmeltTime()
 - ...

```
template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
    template <typename VT>
    VT smelt_many(int n){
        VT container = VT();
        for(int i = 0; i < n; i++){
            container.push_back(smelt());
        }
        return container;
    }
};
```

Template Issue

- 但假設今天的最小需求是很複雜的情況？
- 例如：
 - 你丟進去的型態必須要支援 Smelt()
 - 你丟進去的型態必須要支援 Count()
 - 你丟進去的型態必須要支援 GetSmeltTime()
 - ...
- 在我們能夠理解什麼東西能丟進去時，我們就得要先看一次 Stack 的實作細節
 - 這樣不會很複雜嗎？

```
template <typename T>
class Stack {
private:
    std::vector<T> vec;
public:
    void push(T ore){
        vec.push_back(ore);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        T ore = vec.back();
        vec.pop_back();
        return ore.Smelt();
    }
    template <typename VT>
    VT smelt_many(int n){
        VT container = VT();
        for(int i = 0; i < n; i++){
            container.push_back(smelt());
        }
        return container;
    }
};
```


Template Issue

- 介面告訴了我們一個有趣的 Principle
- 你的某個類別繼承了這個介面，那麼你應該要實作這個介面的 Function
- 你的這個多型能夠放進去的最小需求，是你丟進去的類別應該要實作某些 Function
- 那麼因為我不知道這個多型能夠放進去的最小需求是什麼，不過我可以知道的是他應該要有一些 Function
- 我們也許可以把多型的通用型別換成介面？



Template Issue

- 因此，我們先創一個介面叫做 ISmelttable

```
#include <string>

class ISmelttable {
    virtual std::string Smelt() = 0;
};
```

Template Issue

- 接下來我們改寫這個 Stack
- 這樣有什麼好處?
 - 可以知道放入這個 Stack 的東西一定要是 ISmelttable 類別
 - 那麼 ISmelttable 類別幫我們制定了一定要有 Smelt() 這個函式
- 假設今天這個熔爐的需求變多
 - 我們可以看介面需要我們定義哪些 Function 來知道熔爐最小需求是什麼
 - 我們可以不用看熔爐的實作細節來知道 T 應該要滿足些什麼東西
 - 對於比較廣泛的東西我們再用泛型就好

```
#include <vector>
#include <string>

class ISmelttable {
public:
    virtual std::string Smelt() = 0;
};

class Stack {
private:
    std::vector<std::shared_ptr<ISmelttable>> vec;
public:
    void push(std::shared_ptr<ISmelttable> item){
        vec.push_back(item);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        std::shared_ptr<ISmelttable> item = vec.back();
        vec.pop_back();
        return item->Smelt();
    }
    template <typename VT>
    VT smelt_many(int n){
        VT container = VT();
        for(int i = 0; i < n; i++){
            container.push_back(smelt());
        }
        return container;
    }
};
```

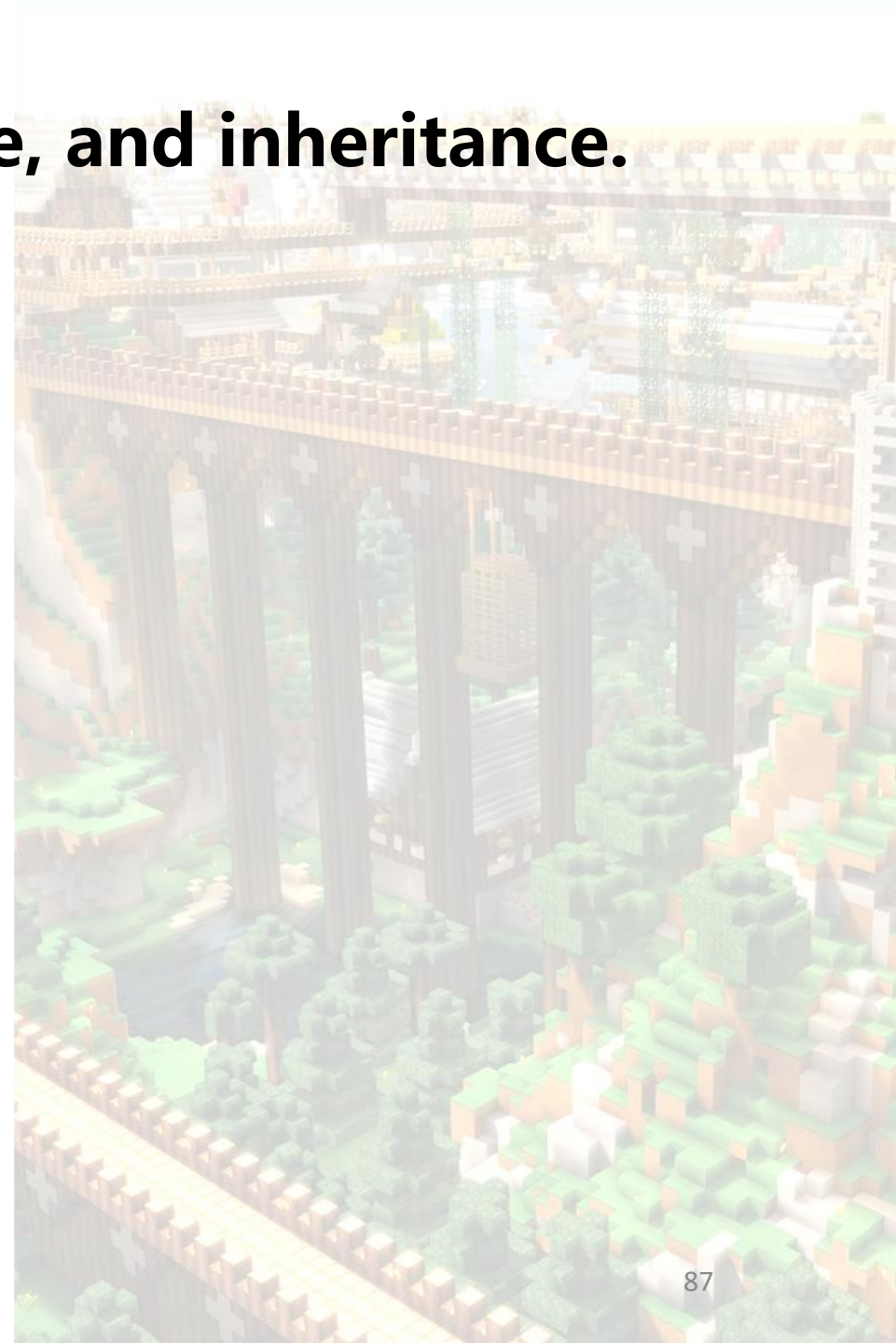



The suggestion

最後，我們想要探討一些有關組合、介面與繼承相關的建議

The suggestion of composition, interface, and inheritance.

- 在聽完前面的課之後，可能發現到我們快要把繼承描述成十惡不赦的大壞蛋了
- 但繼承真的有那麼壞嗎？



The suggestion

- 假設我們想要建立一個 Minecraft API
 - 讓使用者能夠繼承一些類別來創立自己的類別
- 例如：我們希望制定一個 Furnace Stack 的類別
 - 使用者可以創立自己的 Furnace Stack

```
class ISmeltable {
public:
    virtual std::string Smelt() = 0;
};

class Stack {
private:
    std::vector<std::shared_ptr<ISmeltable>> vec;
public:
    void push(std::shared_ptr<ISmeltable> item){
        vec.push_back(item);
    }
    void pop(){
        vec.pop_back();
    }
    int size(){
        return vec.size();
    }
    std::string smelt(){
        std::shared_ptr<ISmeltable> item = vec.back();
        vec.pop_back();
        return item->Smelt();
    }
    template <typename VT>
    VT smelt_many(int n){
        VT container = VT();
        for(int i = 0; i < n; i++){
            container.push_back(smelt());
        }
        return container;
    }
};
```


The suggestion

- 假設我們想要建立一個 Minecraft API
 - 讓使用者能夠繼承一些類別來創立自己的類別
- 例如：我們希望制定一個 Furnace Stack 的類別
 - 使用者可以創立自己的 Furnace Stack
- 因為繼承太壞了，所以我們選擇用組合的方式來創立自己的熔爐
 - 於是有了超多的 Redundant Code
 - `int size()` 裡面包了 `stack.size()`，繼承不會發生

```
class CustomStack {  
private:  
    Stack stack;  
public:  
    void push(std::shared_ptr<ISmeltable> item){  
        stack.push(item);  
    }  
    int size(){  
        return stack.size();  
    }  
    /* ... */  
};
```

The suggestion

- 所以組合會有什麼樣的問題？
 - 造成大量的重複程式碼
- 但如果撇除掉這一點，他依然有很多好處：
 - 減少耦合
 - 對於新加入的需求能夠有效適應

```
class CustomStack {  
private:  
    Stack stack;  
public:  
    void push(std::shared_ptr<ISmeltable> item){  
        stack.push(item);  
    }  
    int size(){  
        return stack.size();  
    }  
    /* ... */  
};
```

The suggestion

- 所以，什麼時候該要用繼承？
 - 當你希望設計一個類別，能夠讓其他的專案維護人員來使用，我們同時想要保護這個類別的細節
 - 因為是給別人使用，我們希望可以將一些實作細節抽象化掉，就會需要更高的耦合
 - 例如：遊戲框架
- 如果我們可以簡單將礦石繼承下來做自己的礦石
 - 我們就不用知道礦石應該要有 ISmeltable 這個介面與礦石應該有的屬性來做組合與介面

```
class ISmeltable {  
    virtual std::string Smelt() = 0;  
};  
  
class OreStone : public ISmeltable {  
private:  
    std::string ore;  
public:  
    OreStone(std::string ore){  
        this->ore = ore;  
    }  
  
    std::string Smelt() override {  
        return ore;  
    }  
};  
  
class CustomOreStone : public OreStone {  
public:  
    CustomOreStone(std::string ore) : OreStone(ore){  
  
    }  
};
```


The suggestion

- 如果我們可以簡單將礦石繼承下來做自己的礦石
 - 我們就不用知道礦石應該要有 ISmelttable 這個介面與礦石應該有的屬性來做組合與介面
- 如果我們希望今天創一個自己的礦石，但又不用繼承的話...
 - 我們還得知道原先 OreStone 用了 ISmelttable 這個介面
 - 然後我們得要繼承這個介面，否則 CustomOreStone 不會有 Smelt() 這個 Function
 - 對於要創自己的礦石會極為不便利

```
class ISmelttable {  
    virtual std::string Smelt() = 0;  
};  
  
class OreStone : public ISmelttable {  
private:  
    std::string ore;  
public:  
    OreStone(std::string ore){  
        this->ore = ore;  
    }  
  
    std::string Smelt() override {  
        return ore;  
    }  
};  
  
class CustomOreStone : public ISmelttable {  
private:  
    OreStone oreStone;  
public:  
    CustomOreStone() : oreStone(OreStone(ore: "some ore")){  
    }  
    std::string Smelt() override{  
        return oreStone.Smelt();  
    }  
};
```

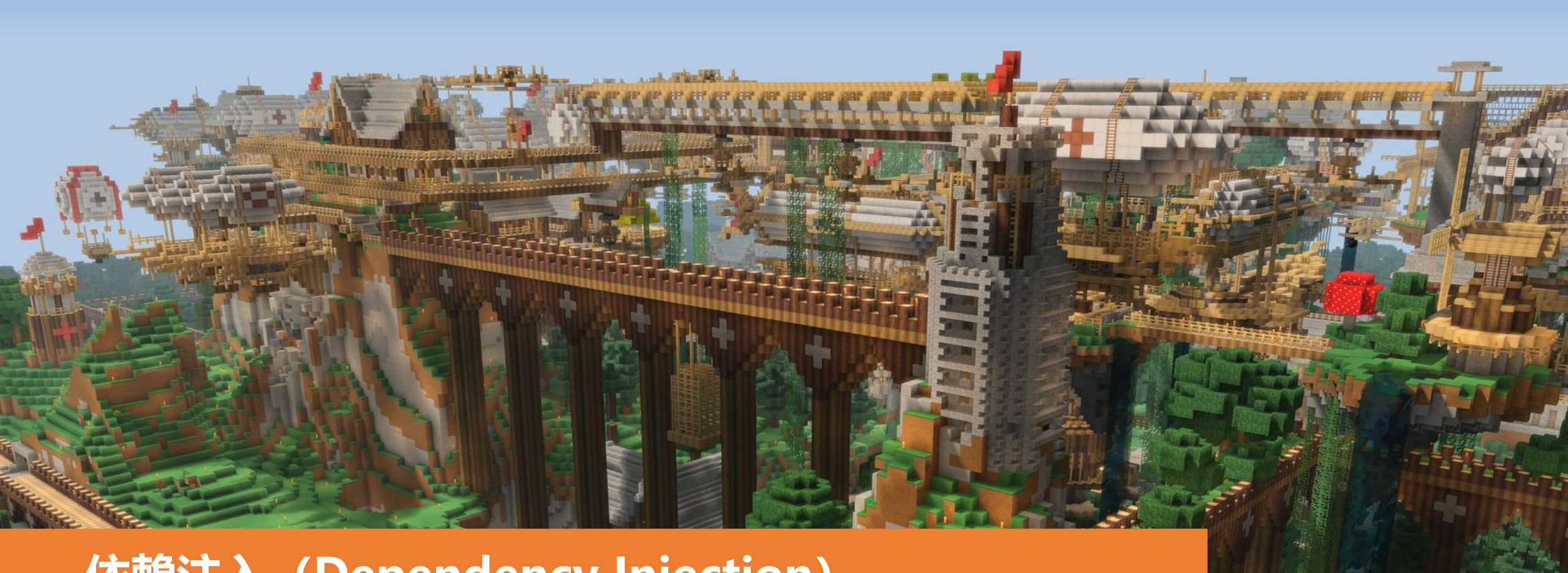
The suggestion

- 如果我們可以簡單將礦石繼承下來做自己的礦石
 - 我們就不用知道礦石應該要有 ISmelttable 這個介面與礦石應該有的屬性來做組合與介面
- 如果我們希望今天創一個自己的礦石，但又不用繼承的話...
 - 我們還得知道原先 OreStone 用了 ISmelttable 這個介面
 - 然後我們得要繼承這個介面，否則 CustomOreStone 不會有 Smelt() 這個 Function
 - 對於要創自己的礦石會極為不便利
- 因此，在這個 Case 上，使用繼承會比較方便
 - 但注意會提高耦合度，但因為我們希望可以抽象化一些實作細節，用繼承就會比較方便

```
class ISmelttable {  
    virtual std::string Smelt() = 0;  
};  
  
class OreStone : public ISmelttable {  
private:  
    std::string ore;  
public:  
    OreStone(std::string ore){  
        this->ore = ore;  
    }  
  
    std::string Smelt() override {  
        return ore;  
    }  
};  
  
class CustomOreStone : public ISmelttable {  
private:  
    OreStone oreStone;  
public:  
    CustomOreStone() : oreStone(OreStone(ore: "some ore")){  
    }  
    std::string Smelt() override{  
        return oreStone.Smelt();  
    }  
};
```

The suggestion

- 因此，組合並不是沒有缺點
 - 在少數情況下，繼承會有更好的可維護性
 - 我們其實可以考慮實作會不會有很頻繁的變更，來決定要不要使用繼承
- 但是在使用組合與介面後，我們可以降低耦合度與對於新的需求有更好的適應力
 - 可以避免掉 Diamond Problem
 - 可以用 Combining Interface 來組出你想要的功能
- 另外，使用繼承時需要注意以下的事情不應該發生：
 - 避免讓子類能夠直接存取父類的值（封裝性）
 - 使用 Protected 來讓子類能夠使用父類的 API
 - 使用 Private 來權限規範好不讓子類使用的函式

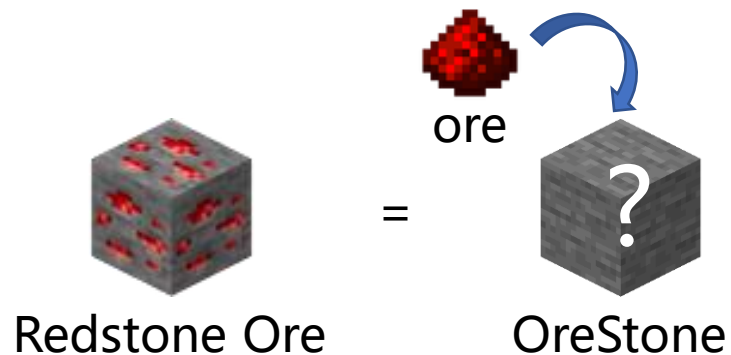


依賴注入 (Dependency Injection)

在我們了解了組合與介面的好處後，我們可以來討論一下依賴注入

依賴注入 (Dependency Injection)

- 什麼叫做依賴注入?
 - 實際上，我們已經看過並使用過了
 - 當我們**在建構子中將類別當作參數**，就是依賴注入的方法
- Ex.
 - 我們改寫成了 ore 是一個礦物的類別
 - 我們在建構子時是傳入一個 Ore 的物件
 - 燒的時候回傳 Ore 類別的物件



```
#include <string>

class Ore {
private:
    std::string name;
public:
    std::string GetName(){
        return this->name;
    }
};

class ISmeltable {
    virtual Ore Smelt() = 0;
};

class OreStone : public ISmeltable {
private:
    Ore ore;
public:
    OreStone(Ore ore){
        this->ore = ore;
    }

    Ore Smelt() override {
        return ore;
    }
};
```


依賴注入 (Dependency Injection)

- 使用 Dependency Injection 有什麼好處？ - (1)
- 基本上我們可以在外部初始化要傳入的類別成員。
 - 如果沒有依賴注入，當我們想要初始化組合裡面的成員，就得要逐一的傳入初始化類別成員所需要的參數。
 - 當我們的類別成員一多的情況下，就會很難維護 Constructor
- 如果我們透過類別物件來初始化的話
 - 非常簡單能夠實現，且非常好維護

```
public:
    OreStone(Ore ore1, Ore ore2, Ore ore3)
        : ore1(ore1),
          ore2(ore2),
          ore3(ore3) {
    }
```

```
class Ore {
private:
    int id;
    std::string name;
public:
    Ore(int id, std::string name){
        this->id = id;
        this->name = name;
    }
    int GetId(){
        return this->id;
    }
    std::string GetName(){
        return this->name;
    }
};

class ISmeltable {
    virtual Ore Smelt() = 0;
};

class OreStone : public ISmeltable {
private:
    Ore ore1;
    Ore ore2;
    Ore ore3;
public:
    OreStone(int id1, std::string name1,
             int id2, std::string name2,
             int id3, std::string name3)
        : ore1( id: id1, name: name1),
          ore2( id: id2, name: name2),
          ore3( id: id3, name: name3) {
    }
};
```


依賴注入 (Dependency Injection)

- 使用 Dependency Injection 有什麼好處? - (2)
- 如果今天的類別成員是一個介面
 - 我們可以透過 DI 來在外部抉擇這個實作應該要是什麼
- 我們基本上可以傳入所有「可以燒的東西」
 - 木頭、礦石、肉...
- 基本上 Furnace 就可以透過呼叫傳入的物件持有的 Smelt 特性，來做不一樣的事情，讓耦合度變低
 - 燒肉可能會飄香
 - 燒礦石可能會比燒木頭、燒礦石還要來得久
 - 取決於傳入的物件，來讓熔爐呼叫 Smelt() 時有不同的特性

```
#include <string>

class ISmeltable {
public:
    virtual std::string Smelt() = 0;
};

class Furnace {
private:
    std::shared_ptr<ISmeltable> item;
public:
    Furnace(std::shared_ptr<ISmeltable> item){
        this->item = item;
    }
    std::string Smelt(){
        return item->Smelt();
    }
};
```

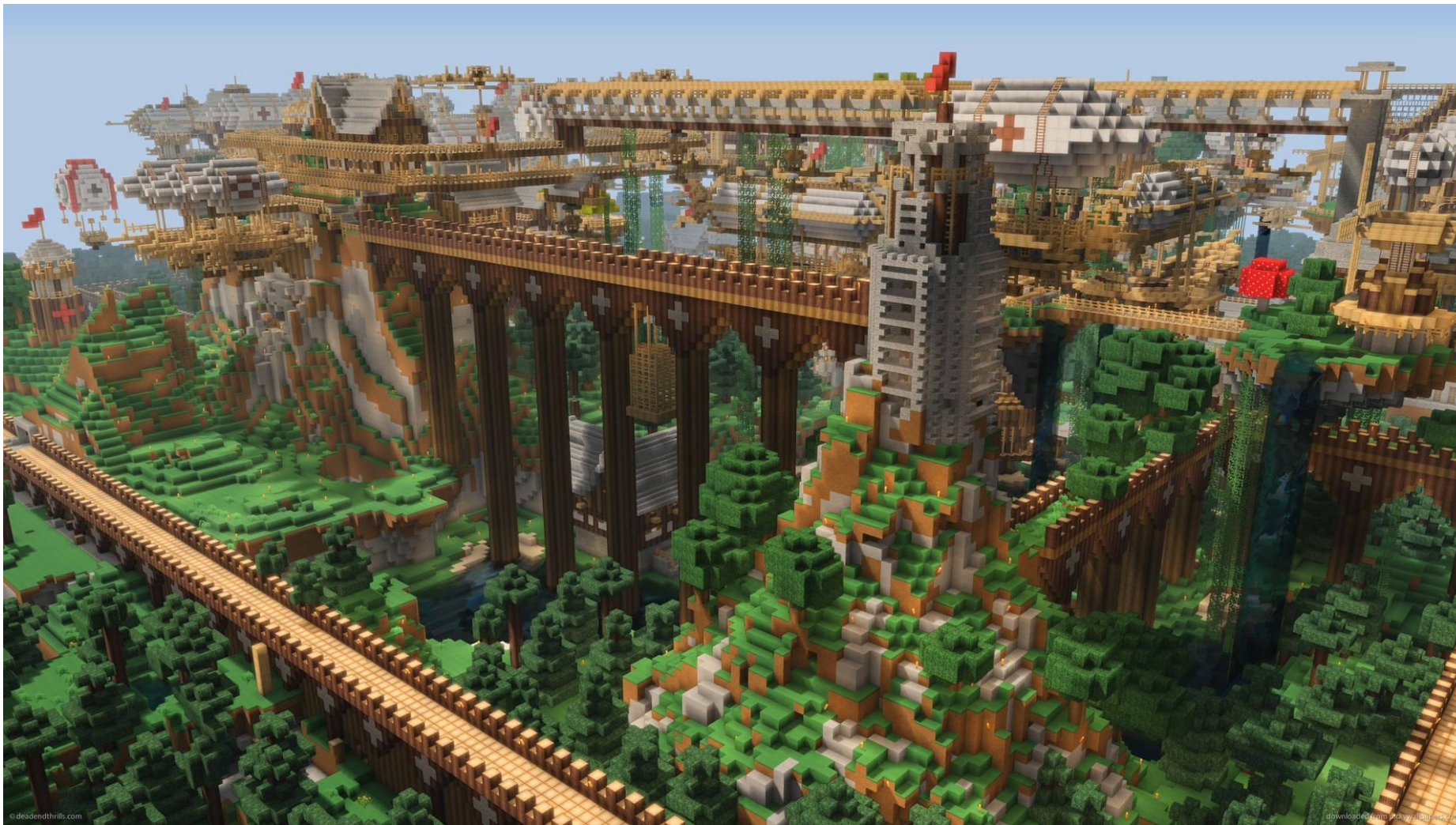
依賴注入 (Dependency Injection)

- 為什麼做這件事情會讓耦合度變低?
 - 因為基本上傳入的物件與熔爐本身無關
- 假設熔爐需要因為傳入的物件是什麼，決定自己要有什麼樣的特性：
 - 耦合度高，我們需要取決於傳入的物件是什麼
 - 不科學
 - 大量的判斷來決定傳入的物件是什麼
 - 如果熔爐希望加上一種新的類別，那麼就要再加上判斷
- 因此透過依賴注入，就能夠減少耦合度
 - 取決於傳入的物件是什麼，才來呼叫 Smelt() 自己有的特性

```
#include <string>

class ISmeltable {
public:
    virtual std::string Smelt() = 0;
};

class Furnace {
private:
    std::shared_ptr<ISmeltable> item;
public:
    Furnace(std::shared_ptr<ISmeltable> item){
        this->item = item;
    }
    std::string Smelt(){
        return item->Smelt();
    }
};
```



結論

總結一下這個章節在講什麼

結論

- 我們討論到了組合的一些特性
 - 聚合 (Aggregation) 與組合 (Composition)
 - 取決於是不是委派給類別。
- 我們討論到了一些組合與介面用來解決繼承的問題
 - 繼承鏈是一個高耦合度的實作，難以維護
 - 透過組合與介面能夠扁平化繼承鏈，降低耦合。
- 組合與介面還能優化掉很多有趣的問題
 - 鑽石問題
 - 透過組合介面來重複利用介面組出想要的東西
 - 優化掉使用泛型時難以理解最小需求的問題

結論

- 我們也討論到了繼承、組合與介面的好與壞
 - 繼承可能更適合用在不需要討論耦合的情況
 - 組合與介面能夠更好的應付需求與降低耦合度
 - 使用繼承時需要額外小心一些問題
- 最後，我們討論到了依賴注入
 - 能夠根據傳入的物件是什麼來做出對應的行為。
 - 例如熔爐丟肉進去燒會飄香、丟礦石可能會燒比較慢...
 - 依賴注入也能有效優化建構子的部分，可以不用傳入組合所需要的所有成員，導致難以維護建構子的參數。